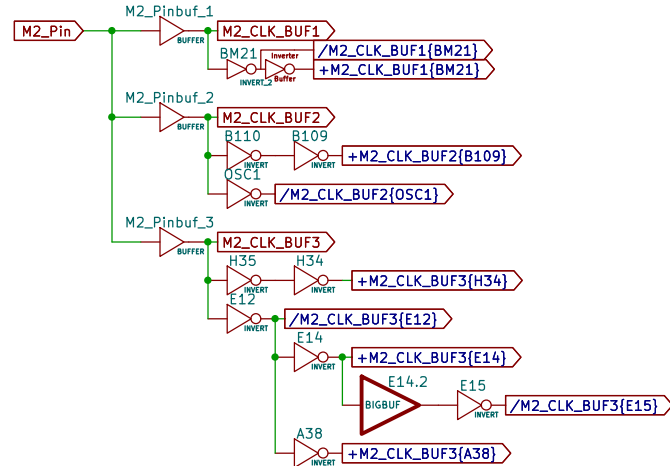
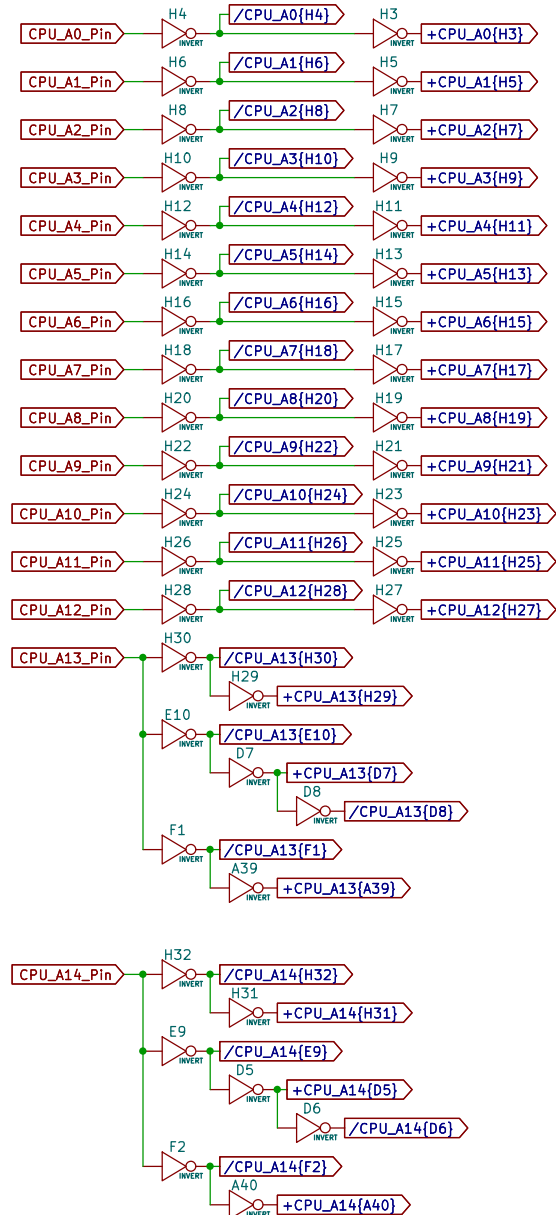


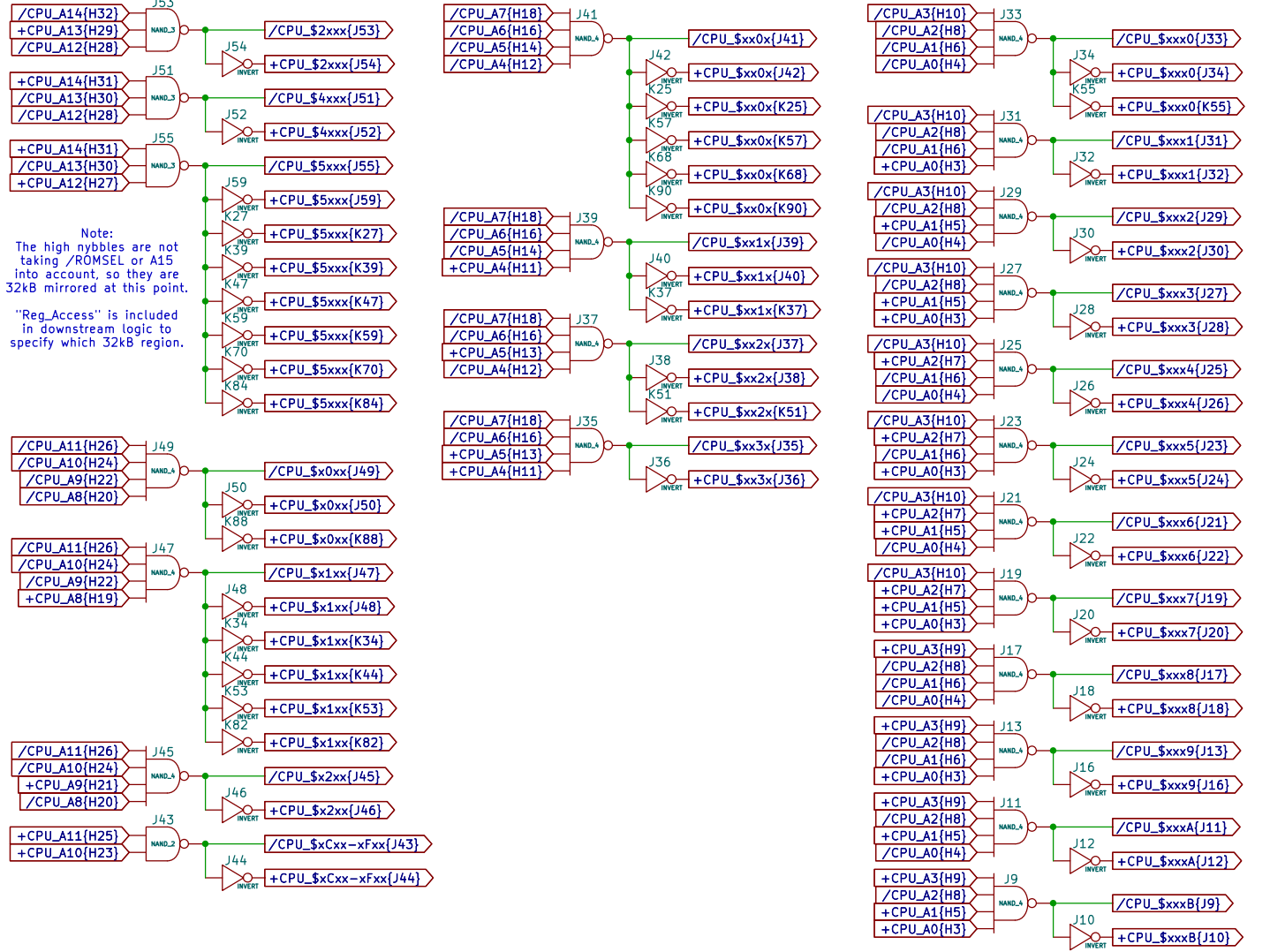
Nametable Mapping	
	File: nametable_mapping.kicad_sch
CPU Address Bus	
	File: cpuAddressBus.kicad_sch
Register Access	
	File: register_access.kicad_sch
PRG Banking	
	File: prg_banking.kicad_sch
PPU Address Bus	
	File: ppu_address_bus.kicad_sch
Reset Detection	
	File: reset_detection.kicad_sch
CPU Data Bus	
	File: cpu_data_bus.kicad_sch
PCM DAC	
	File: pcm_dac.kicad_sch
Scanline Detection	
	File: scanline_detection.kicad_sch
V-Split	
	File: v_split.kicad_sch
PPU Fetch Counting	
	File: ppu_fetch_counting.kicad_sch
CHR A0,A1,A2	
	File: chr_a0_a1_a2.kicad_sch
ExRAM Mode	
	File: exram_mode.kicad_sch
Fill Mode	
	File: fill_mode.kicad_sch
Pulse Length Lookup Table	
	File: pulse_length_lookup_table.kicad_sch
Pulse Timer	
	File: pulse_timer.kicad_sch
Pulse Duty Envelope Volume	
	File: pulse_duty_envelope_volume.kicad_sch
Pulse DAC	
	File: Pulse DAC.kicad_sch
PPU Data Bus	
	File: ppu_data_bus.kicad_sch
CHR Bank Registers	
	File: chr_bank_registers.kicad_sch
CHR Bank Selection Logic	
	File: chr_bank_selection_logic.kicad_sch
CHR Address Bus	
	File: chr_address_bus.kicad_sch



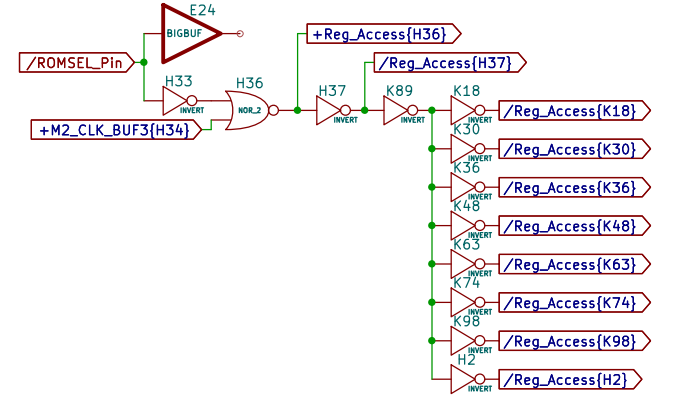
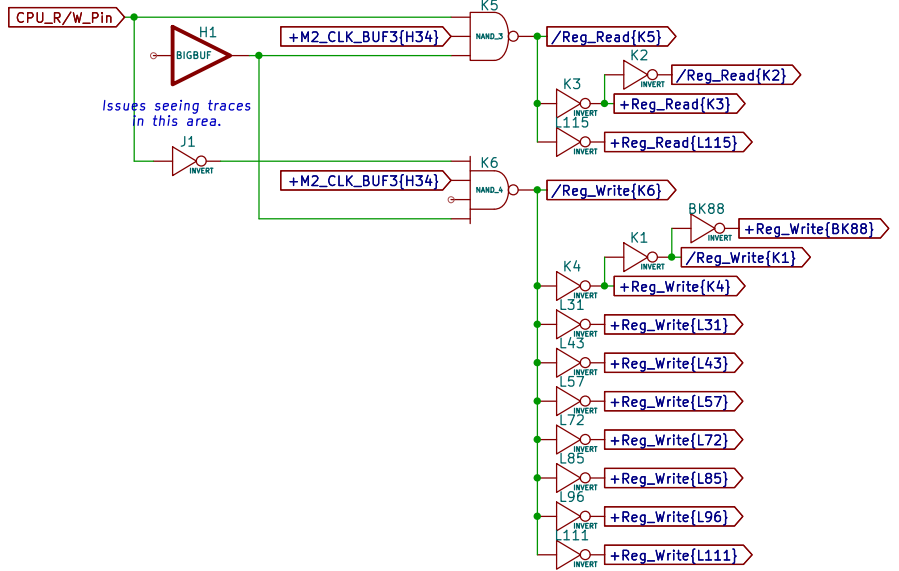
CPU Address Bus Buffering

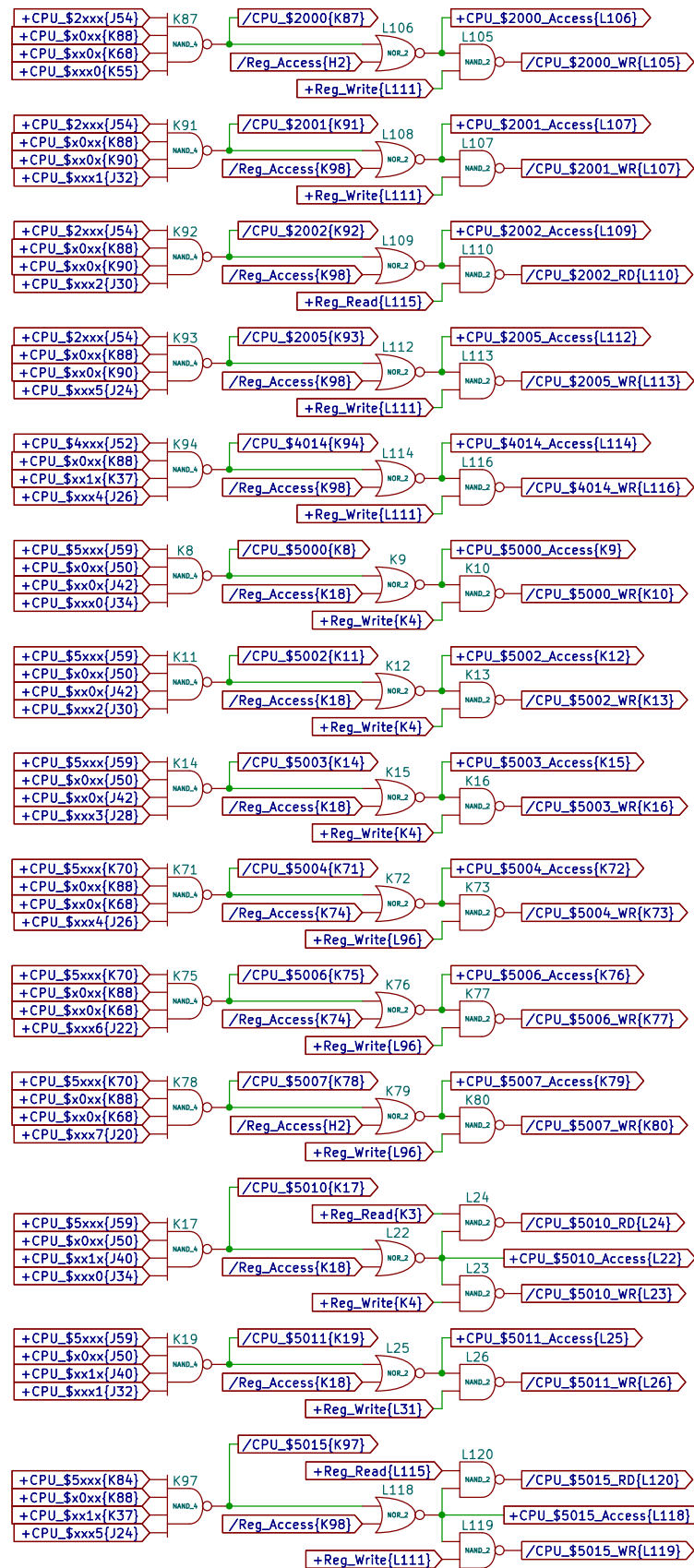


CPU Address Bus Nybble Decoding

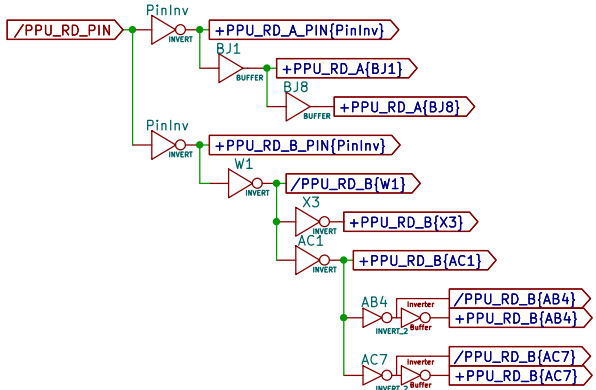
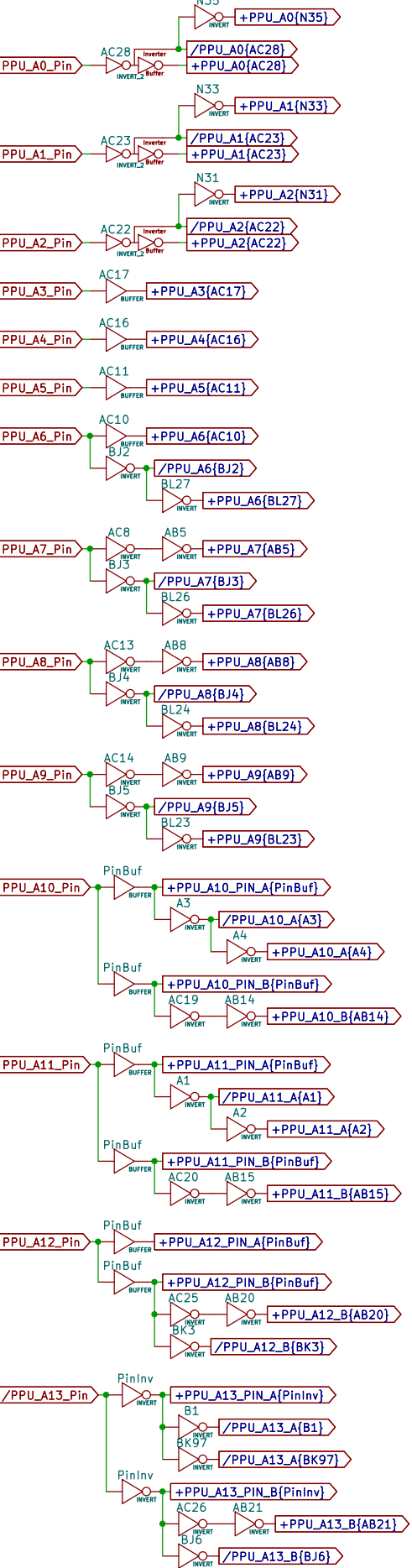


Note:
The high nybbles are not taking /ROMSEL or A15 into account, so they are 32kB mirrored at this point.
"Reg_Access" is included in downstream logic to specify which 32kB region.

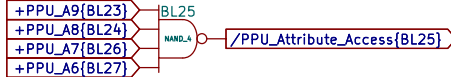


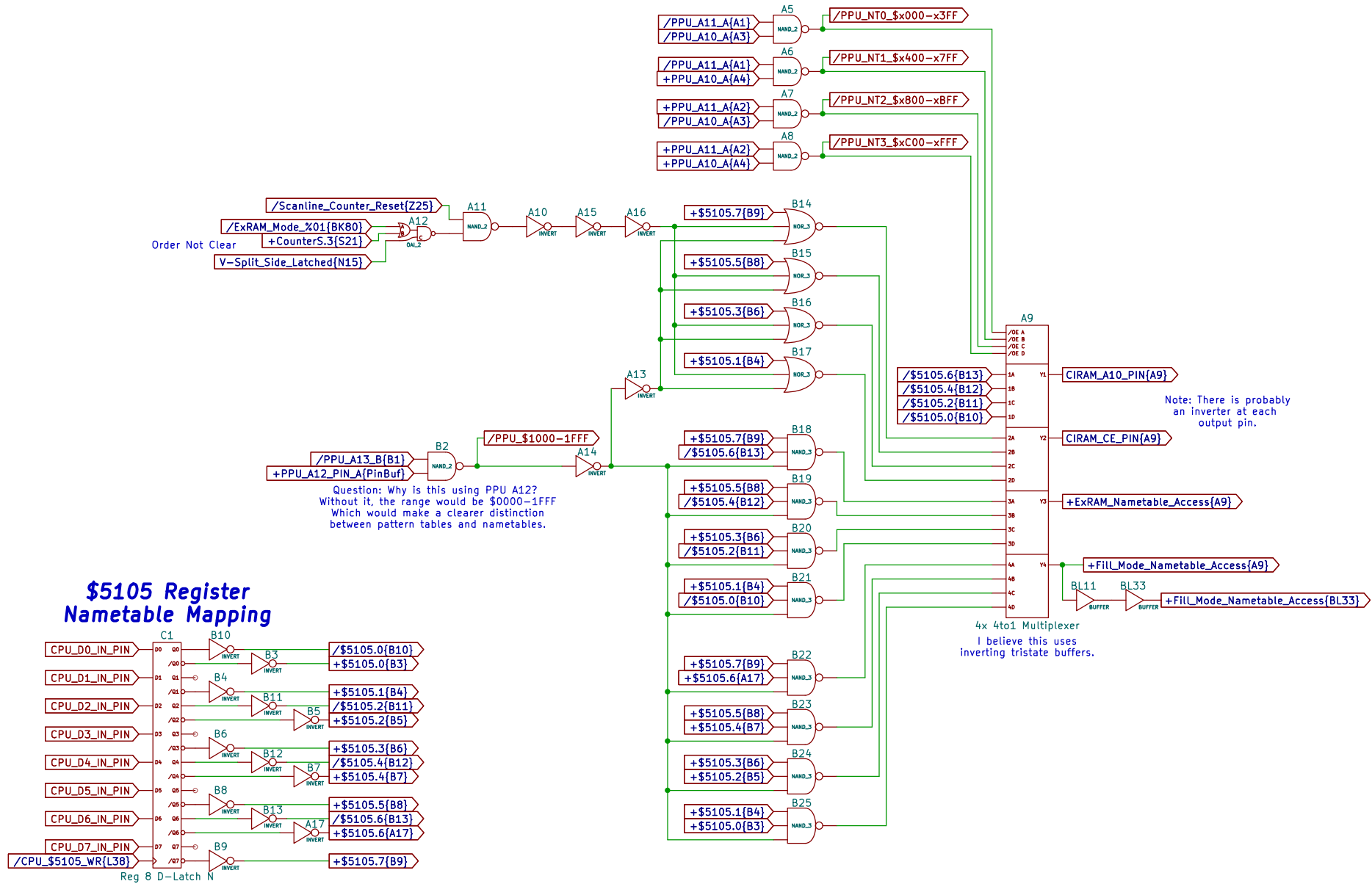


PPU Address Bus Buffering



Note:
/PPU_WR_PIN has an inverter
at the pin but does not go
to any buffers beyond that.





Reg 4 D-Latch

Diagram of the Reg 8 D-Latch. The register has 8 inputs (B40 to B47) and 8 outputs (B40 to B47). The inputs are connected to a bus labeled B40. The outputs are connected to a bus labeled B40. The register is labeled 'Reg 8 D-Latch'.

The diagram shows an 8 D-Latch (B39) with the following connections:

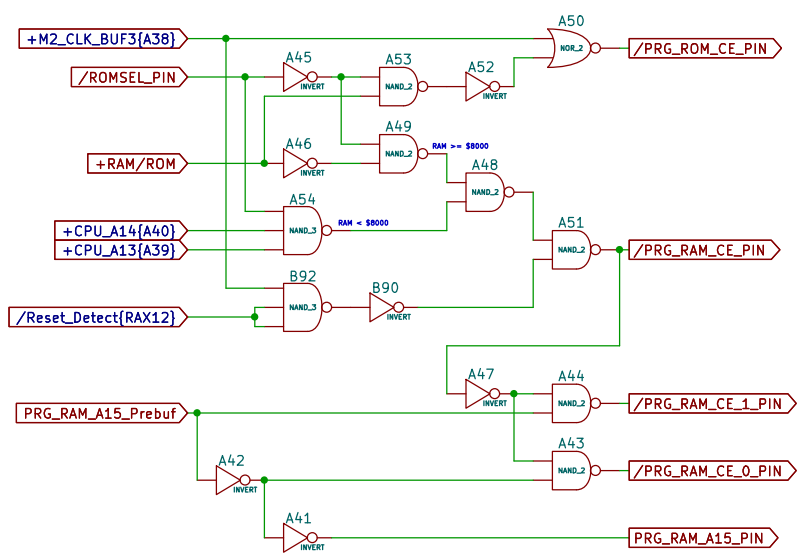
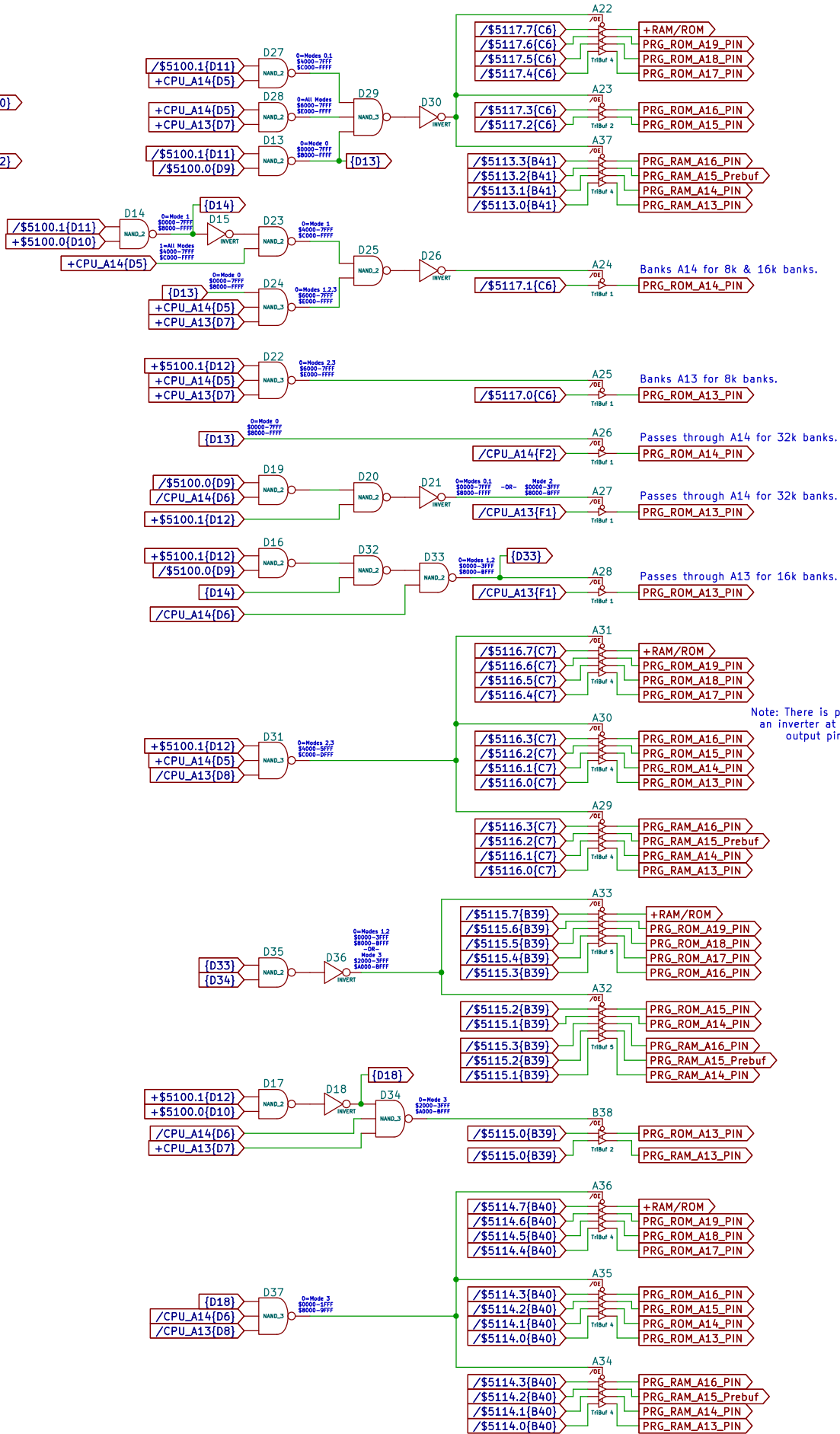
- Inputs (D0-D7):**
 - D0: /CPU_D0[E3]
 - D1: /CPU_D1[E4]
 - D2: /CPU_D2[E1]
 - D3: /CPU_D3[E2]
 - D4: /CPU_D4[E5]
 - D5: /CPU_D5[E7]
 - D6: /CPU_D6[E6]
 - D7: /CPU_D7[E8]
- Outputs (Q0-Q7):**
 - Q0: \$5115.0[B39]
 - Q1: \$5115.1[B39]
 - Q2: \$5115.2[B39]
 - Q3: \$5115.3[B39]
 - Q4: \$5115.4[B39]
 - Q5: \$5115.5[B39]
 - Q6: \$5115.6[B39]
 - Q7: \$5115.7[B39]
- Control:**
 - Enable:** /CPU_5115_WR[L49]
 - Label:** Reg 8 D-Latch

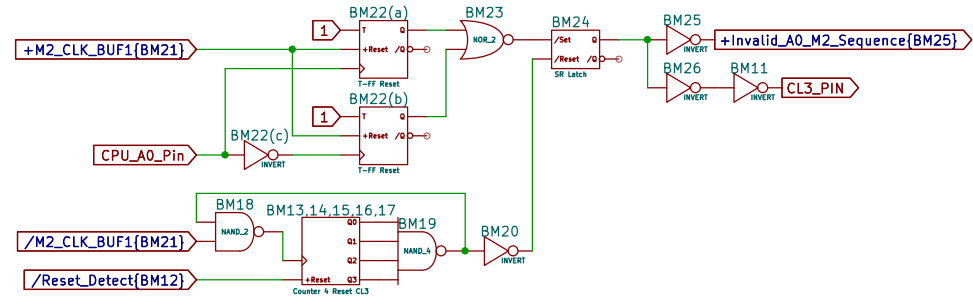
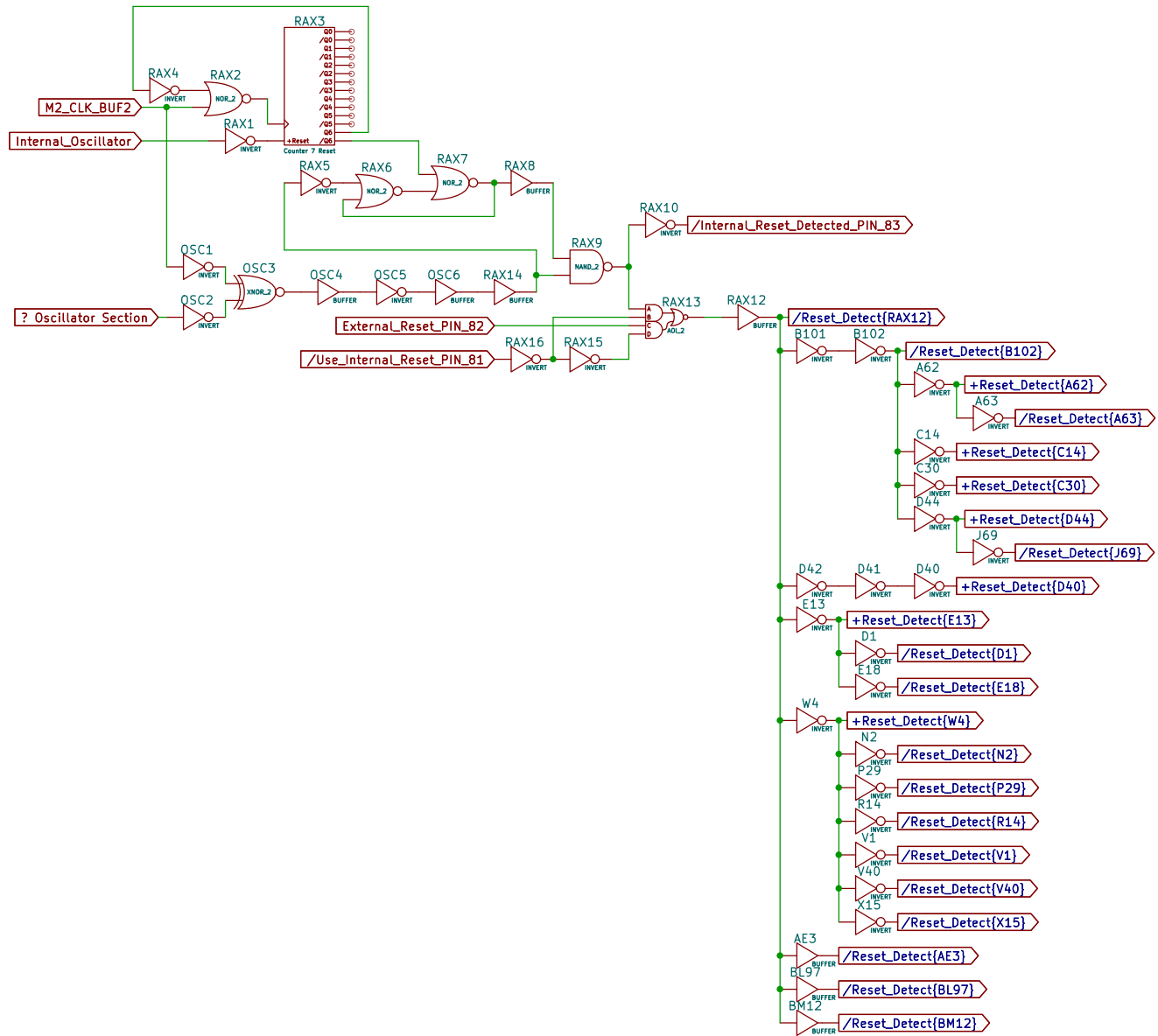
Diagram of the Reg 8 D-Latch. The latch has 8 data inputs (D0-D7) and one enable input (C7). The data inputs are connected to CPU_D0[E3] through CPU_D7[E8]. The enable input C7 is connected to CPU_D0[E3]. The output of the latch is CPU_5116_WR[L52].

Diagram illustrating the C6 register file structure and connections:

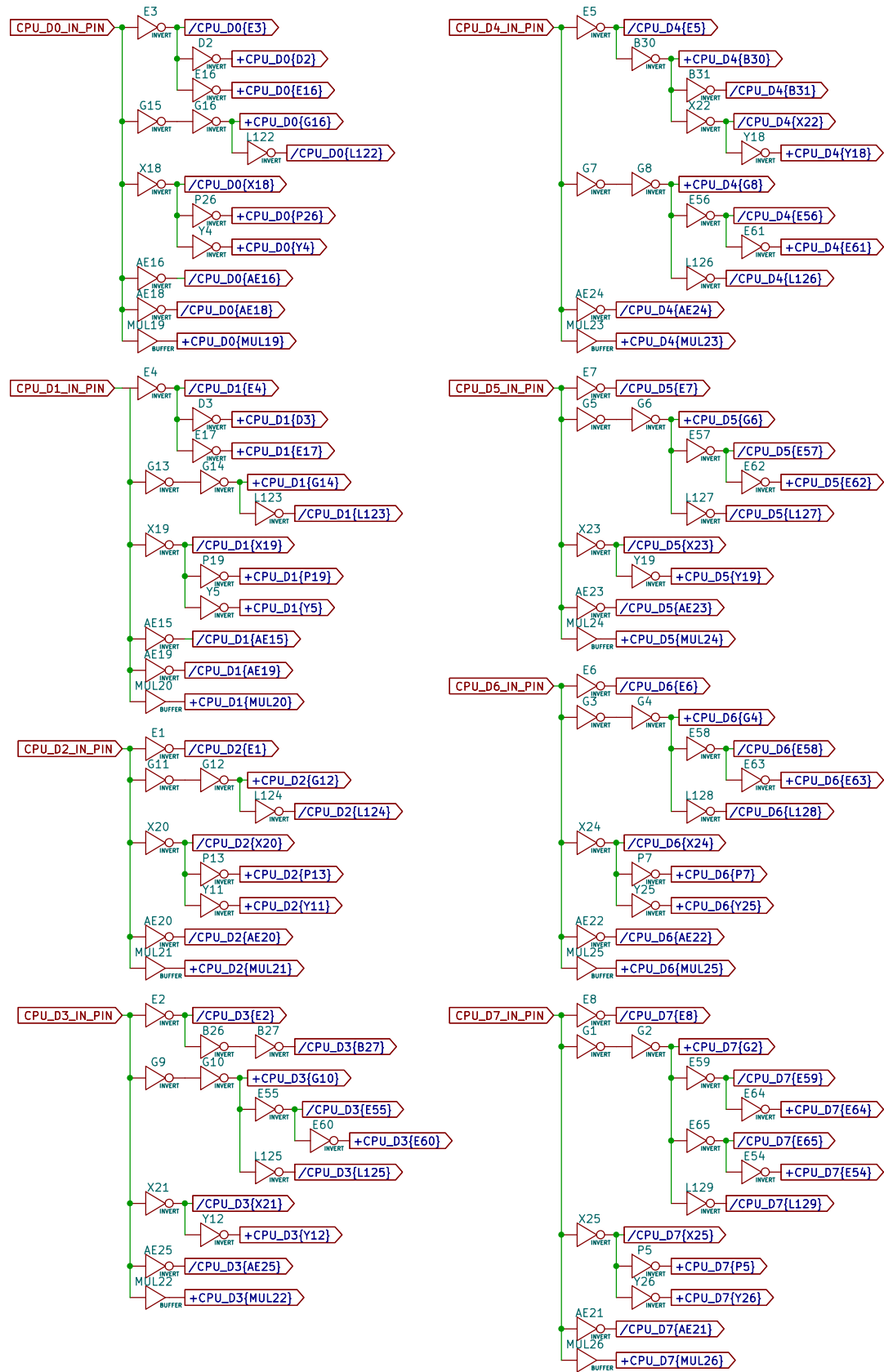
- Registers D0 through D7 are shown, each connected to a specific RAM location (\$5117.0[C6] through \$5117.7[C6]).
- Register D7 is also connected to +Reset_Detect[E13] and +Set.
- Register D7 is labeled as Reg 8 DFF-Set.
- Note: \$5117.7 RAM is fully implemented.

Note:
\$5117.7 RAM/ROM bit
is fully implemented
but is somehow being
prevented from entering
RAM mode when bench
testing.

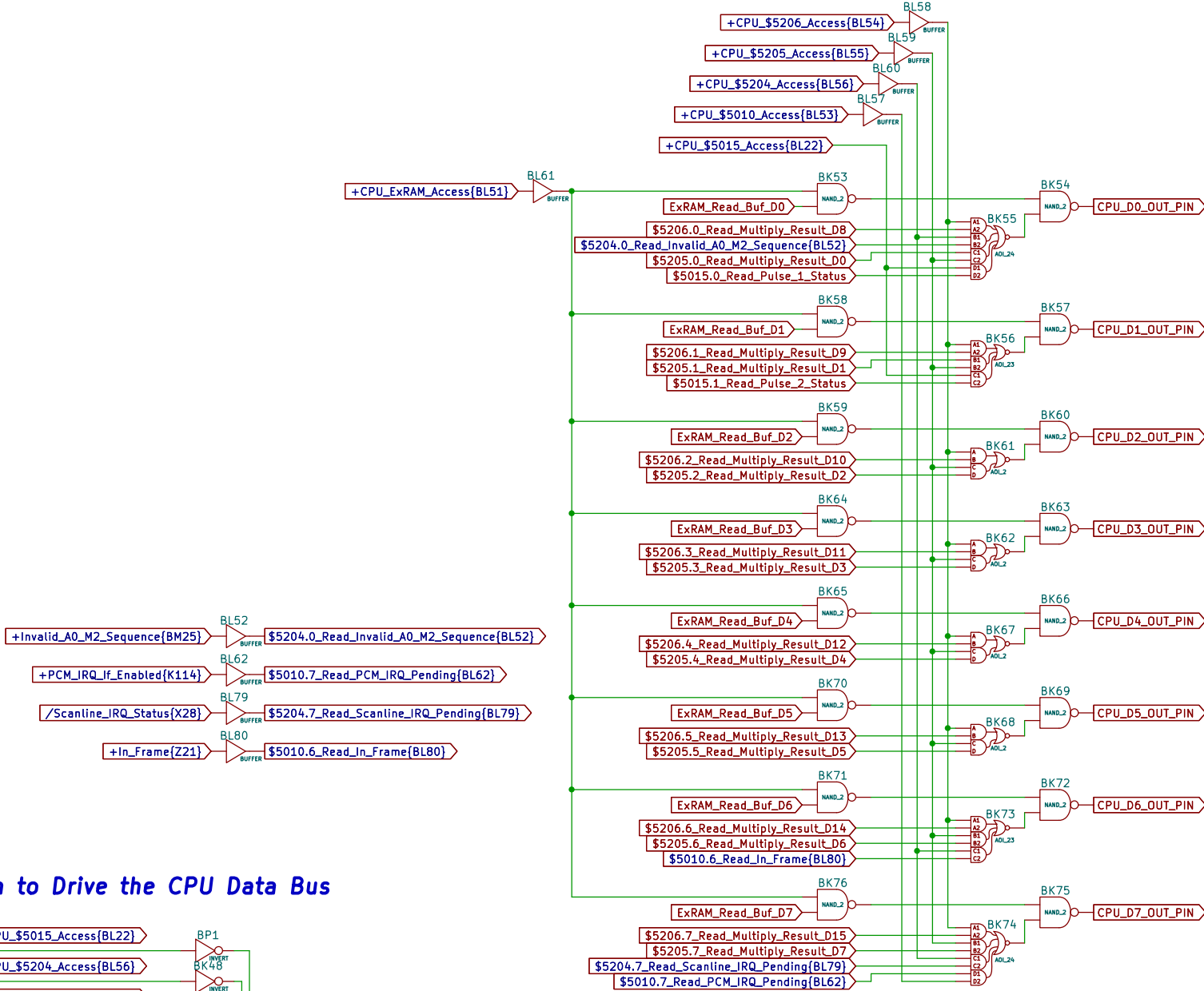




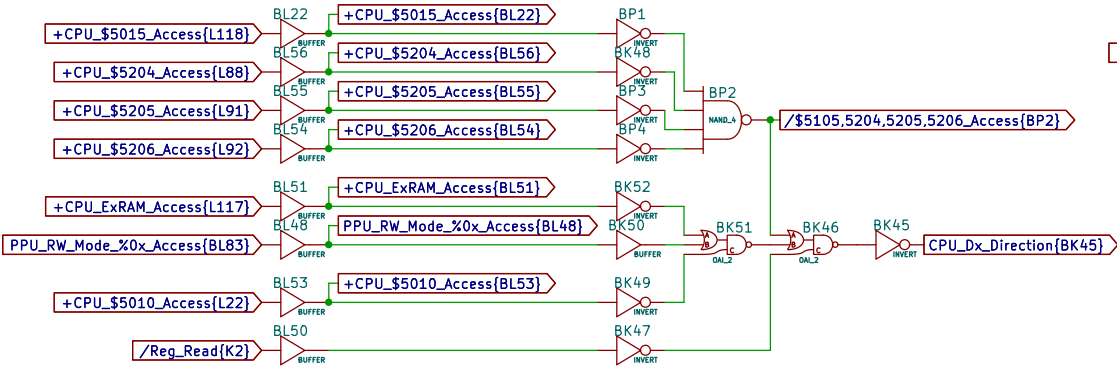
Incoming CPU Data Bus Buffering Bonanza



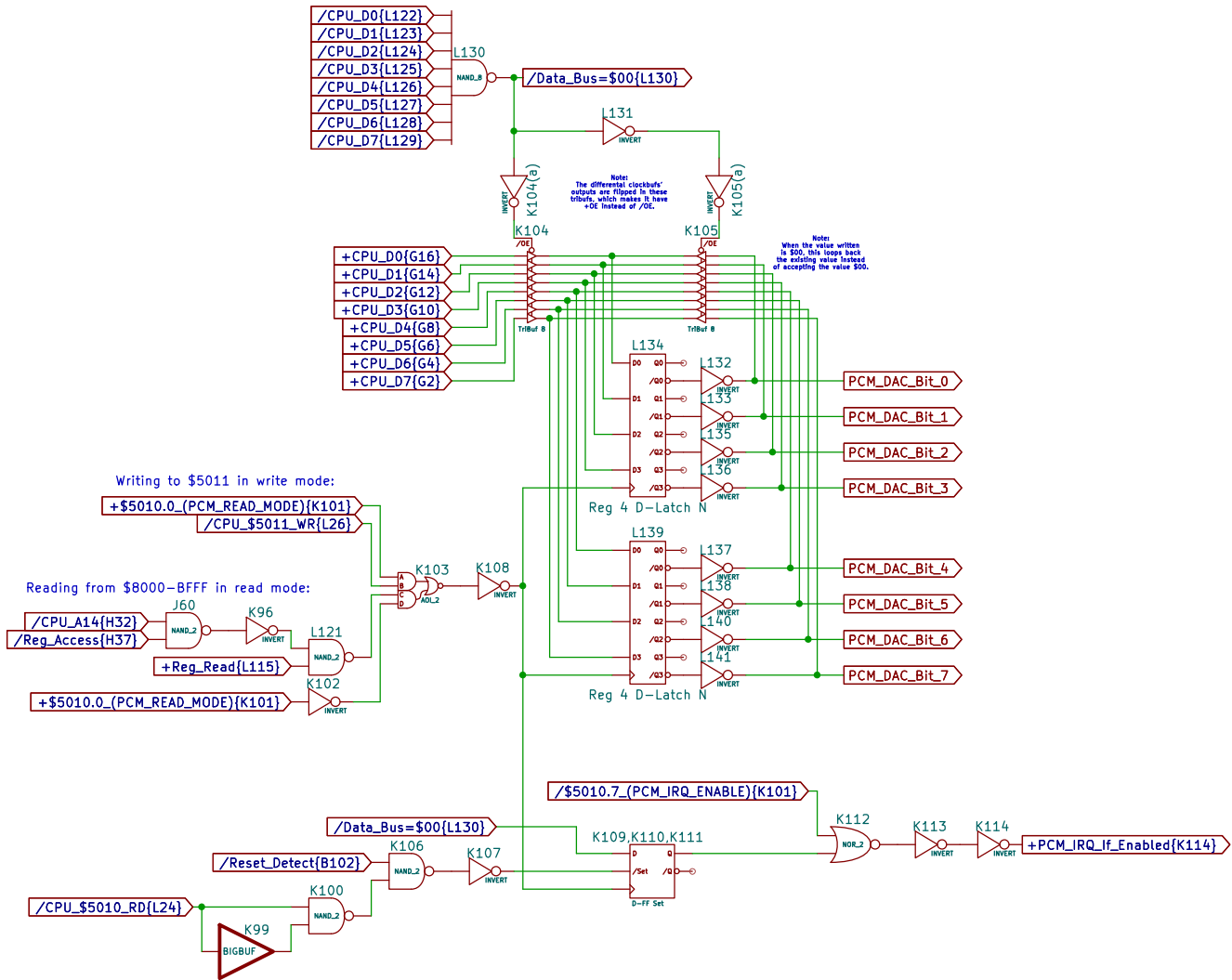
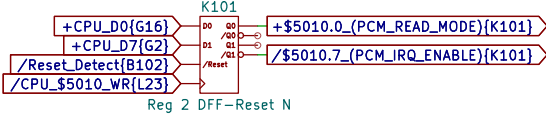
Multiplexing Data to be Driven onto the CPU Data Bus

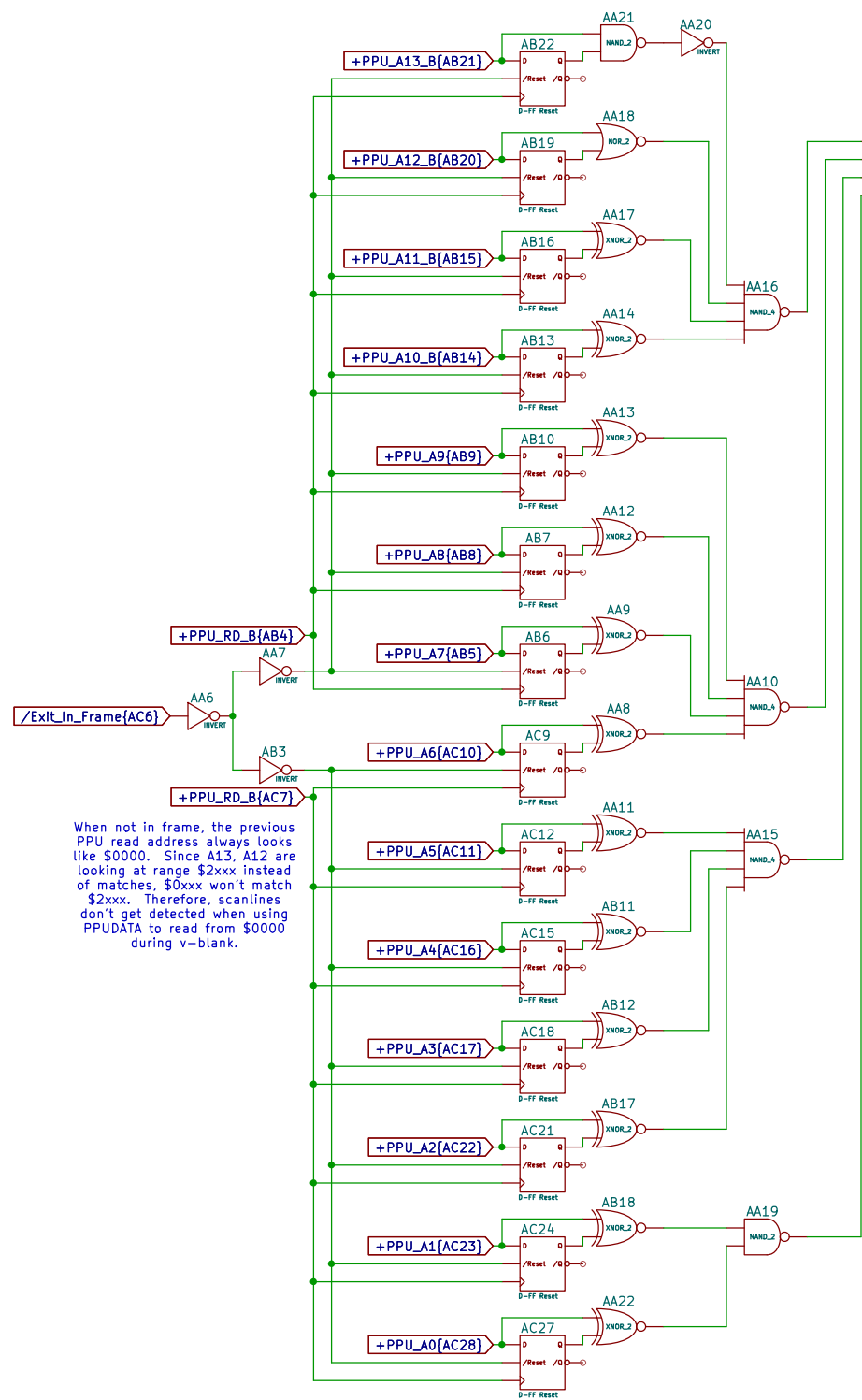


Decides When to Drive the CPU Data Bus

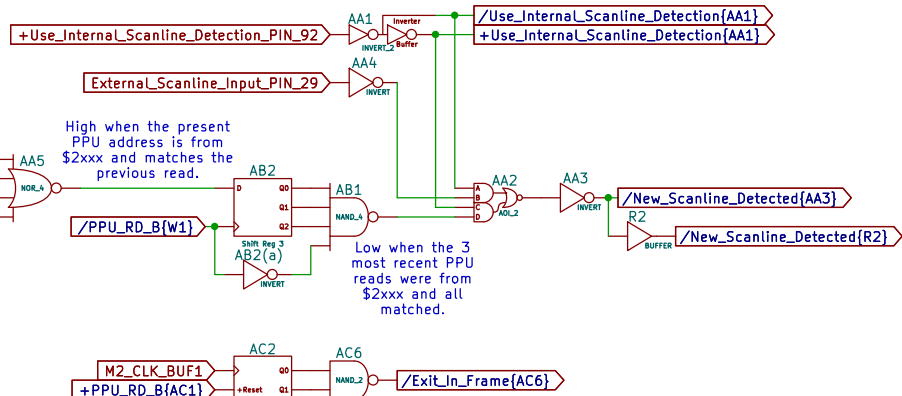


**\$5010 Register
Mode & IRQ Enable**

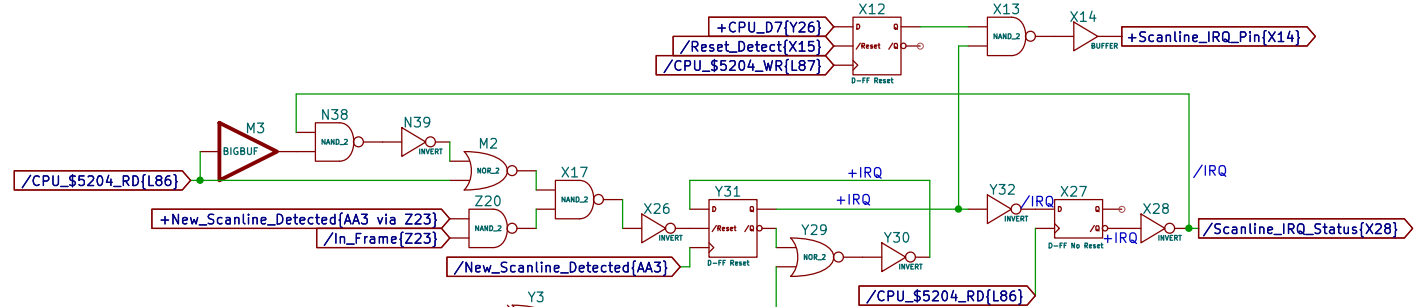




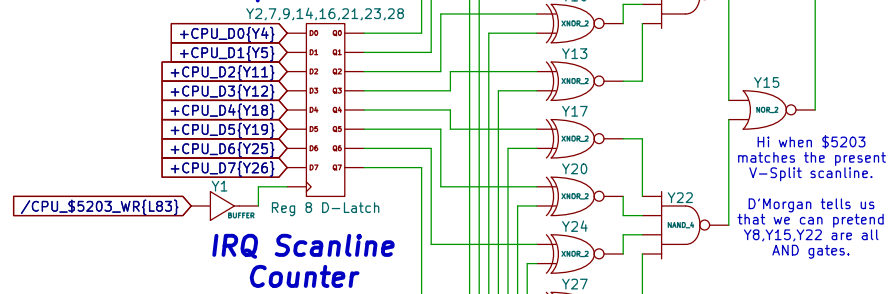
When not in frame, the previous PPU read address always looks like \$0000. Since A13, A12 are looking at range \$2xxx instead of matches, \$0xxx won't match \$2xxx. Therefore, scanlines don't get detected when using PPU_DATA to read from \$0000 during v-blank.



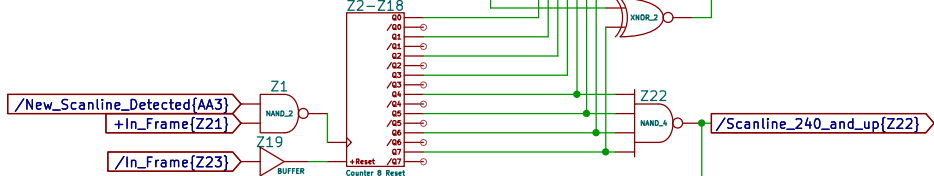
\$5204 Register Scanline IRQ Enable



\$5203 Register IRQ Scanline Compare Value

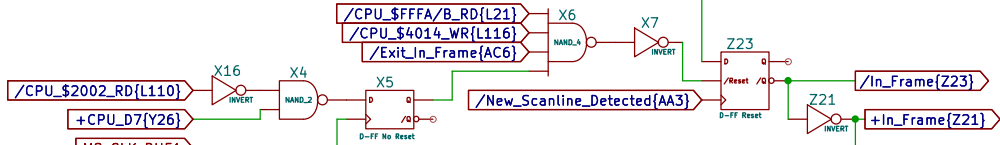


IRQ Scanline Counter



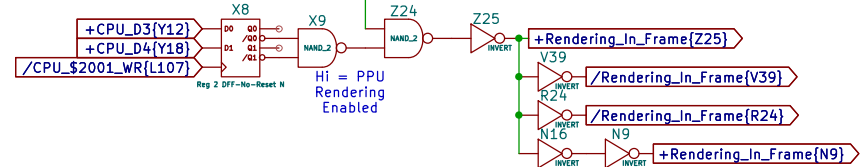
\$2002 Register Read Snooping

Latches \$2002.7 value read by CPU for 1 CPU cycle.

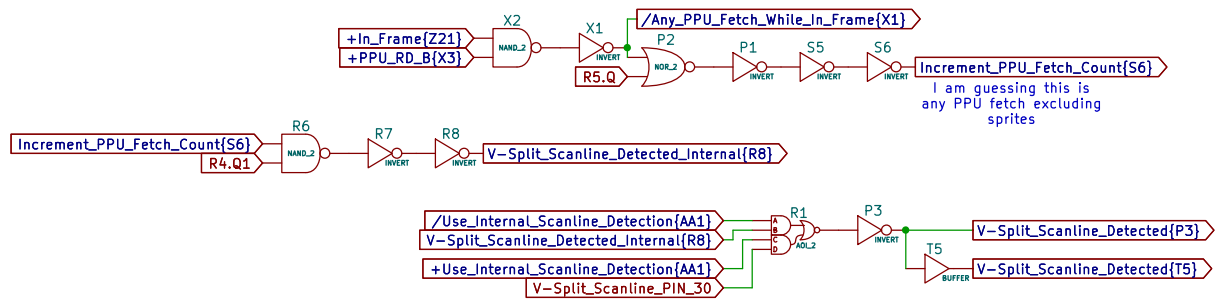
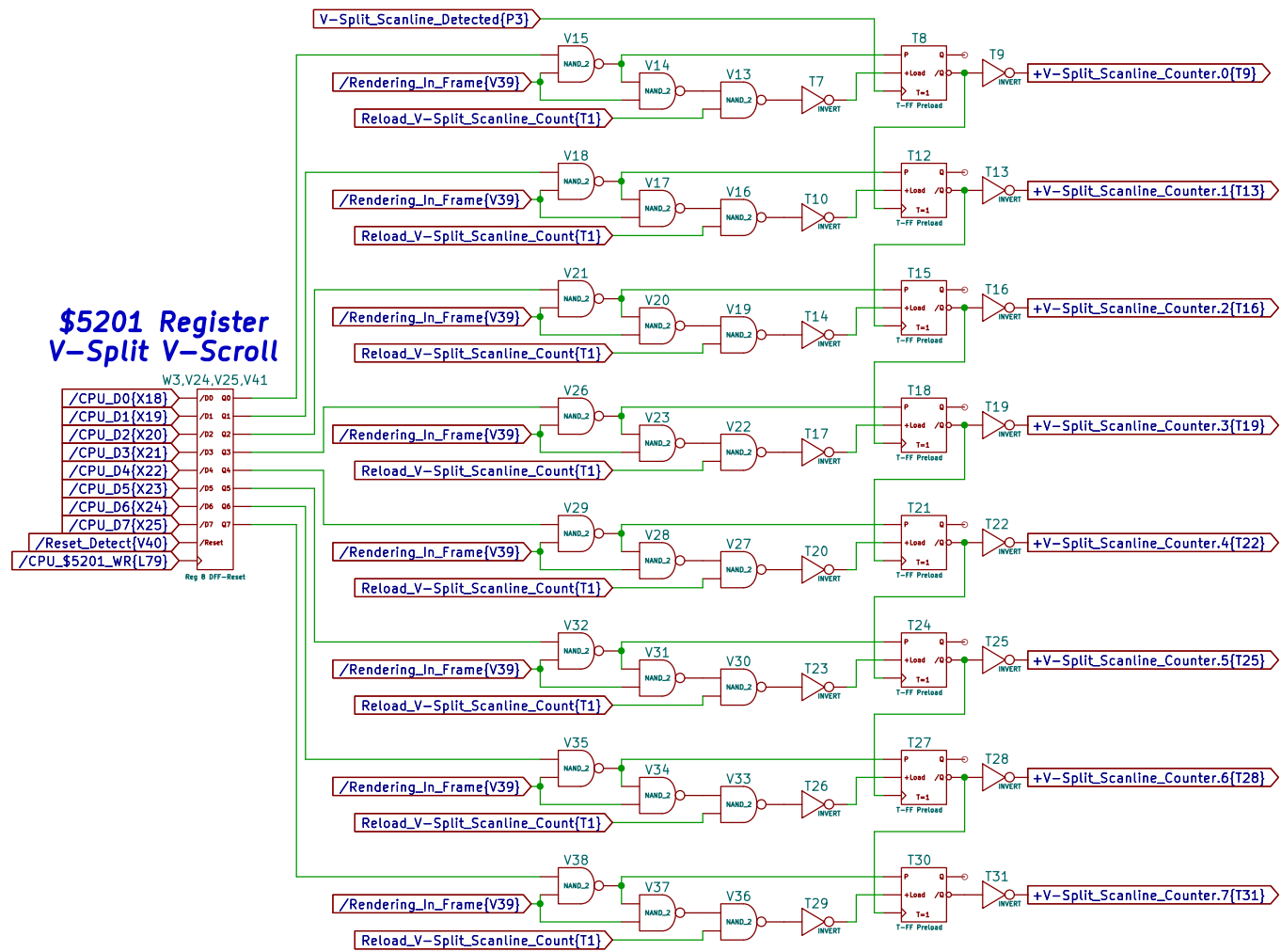


\$2001 Register Write Snooping

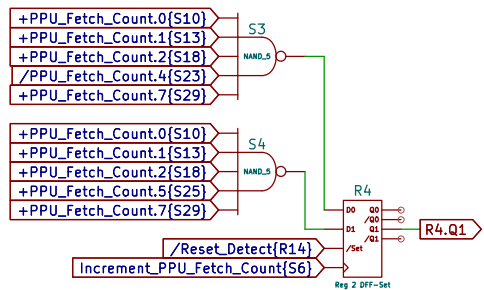
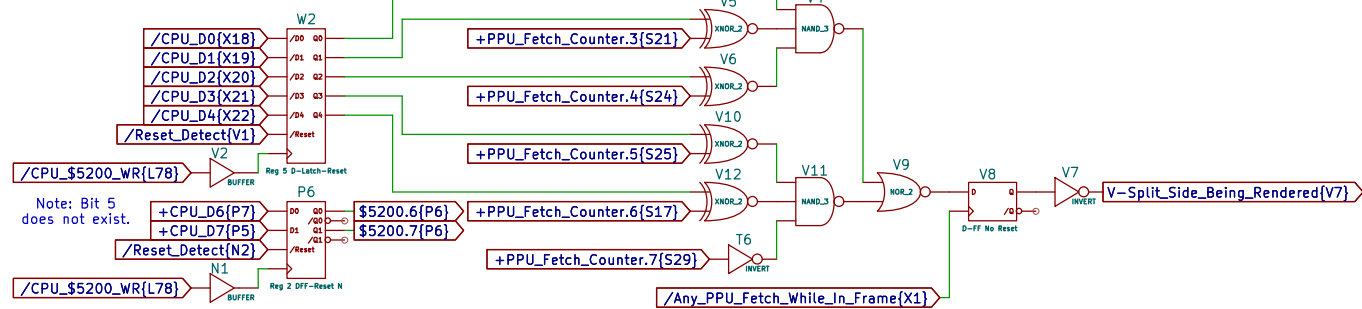
Hi = PPU Rendering Enabled



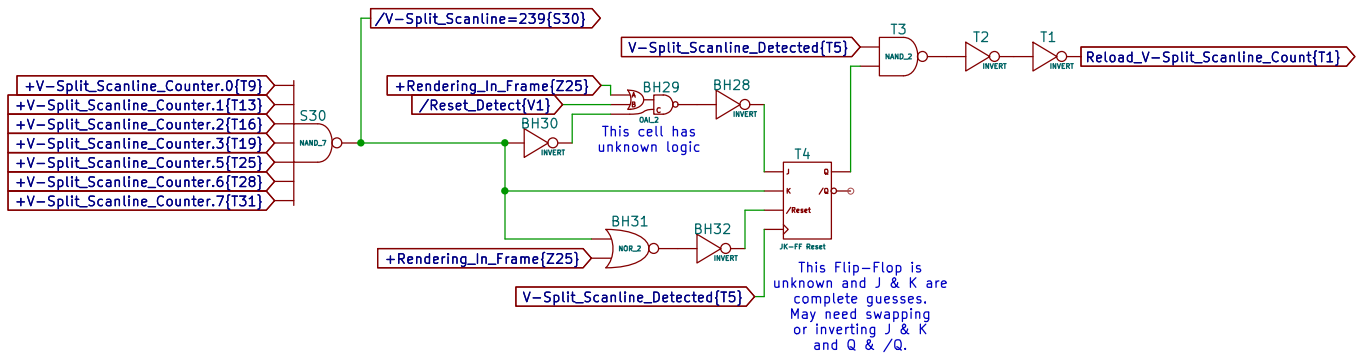
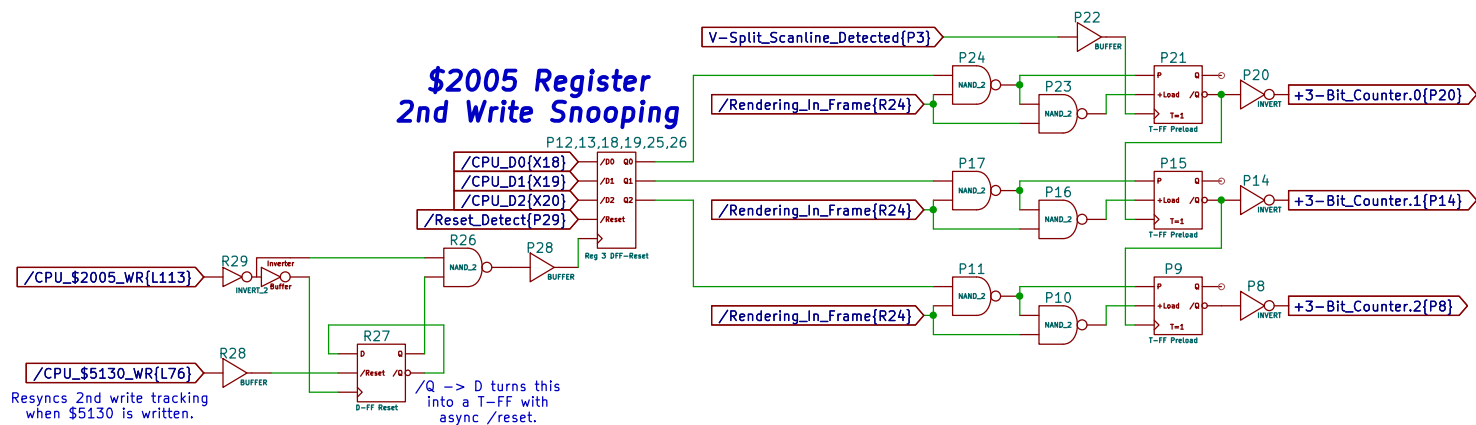
V-Split Scanline Counter



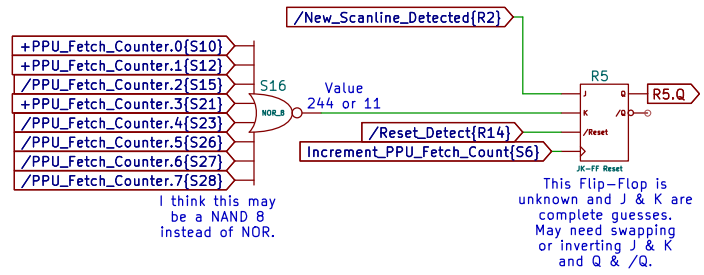
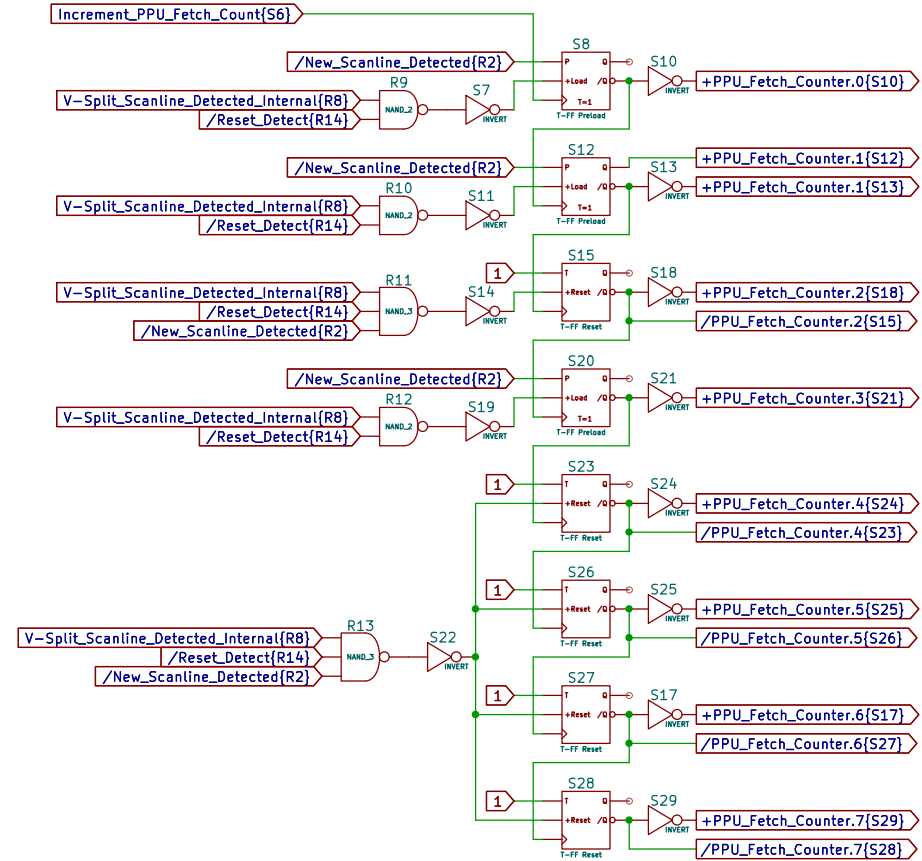
\$5200 Register V-Split Mode & H-Threshold

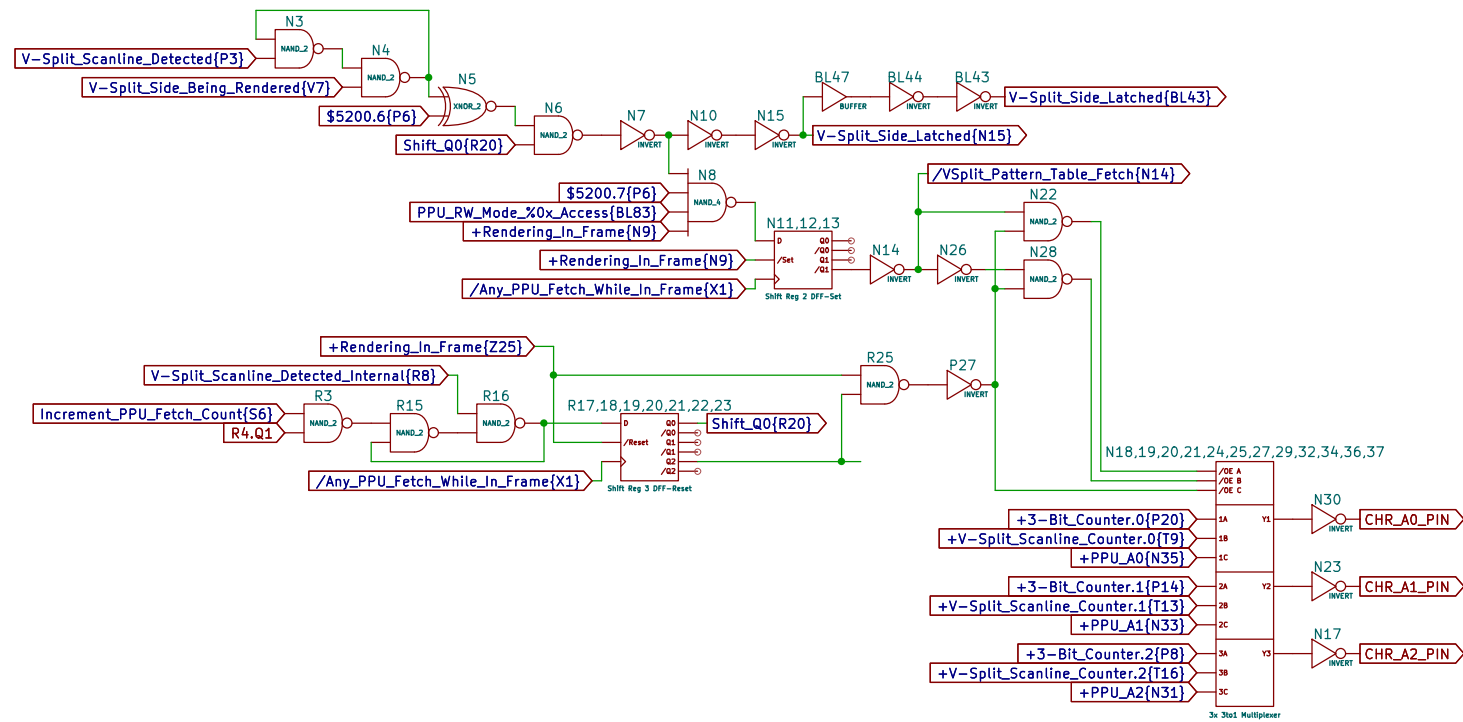


Unknown 3-Bit Scanline Counter

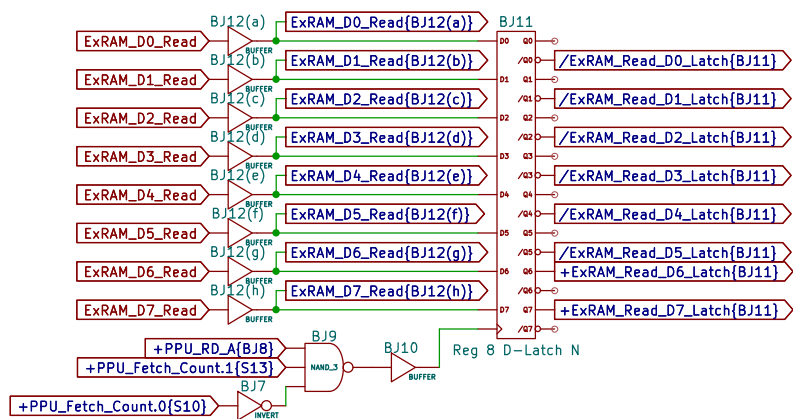
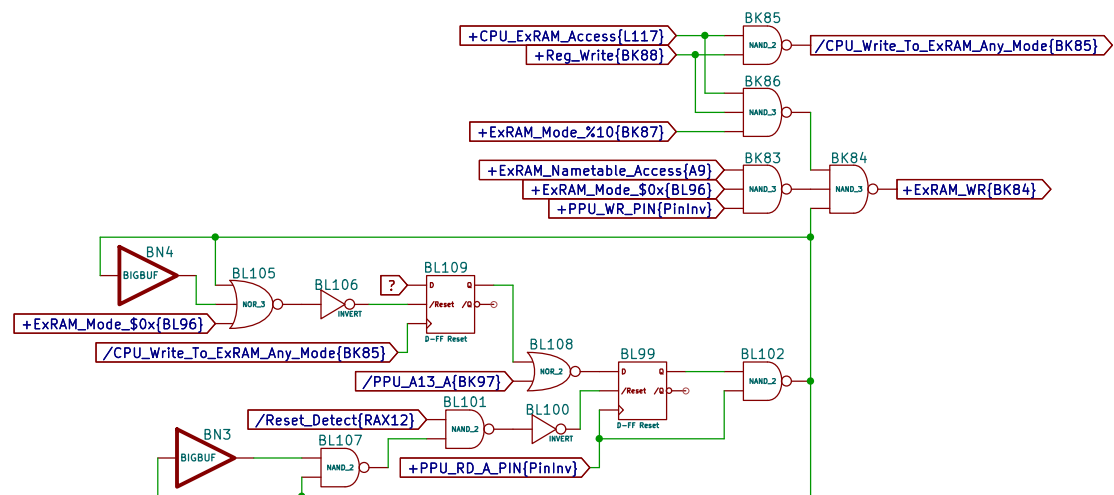
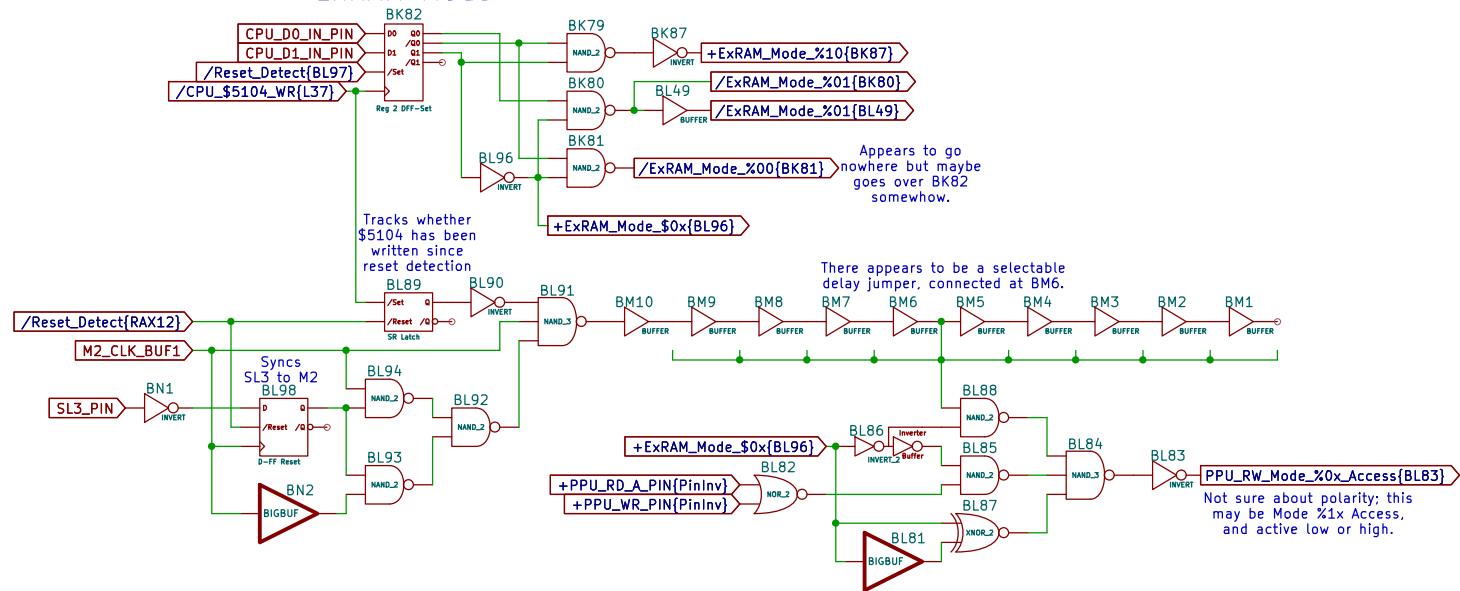


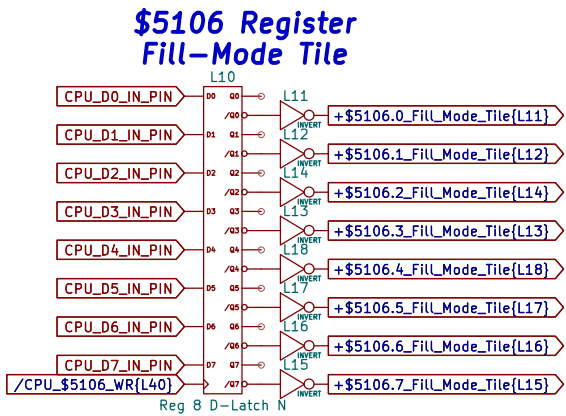
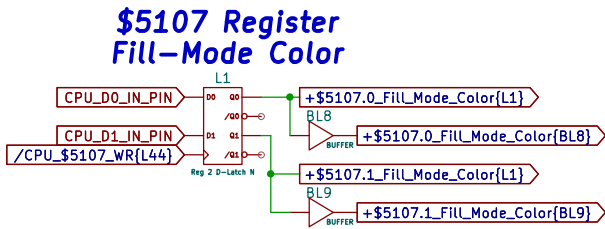
PPU Fetch Counter



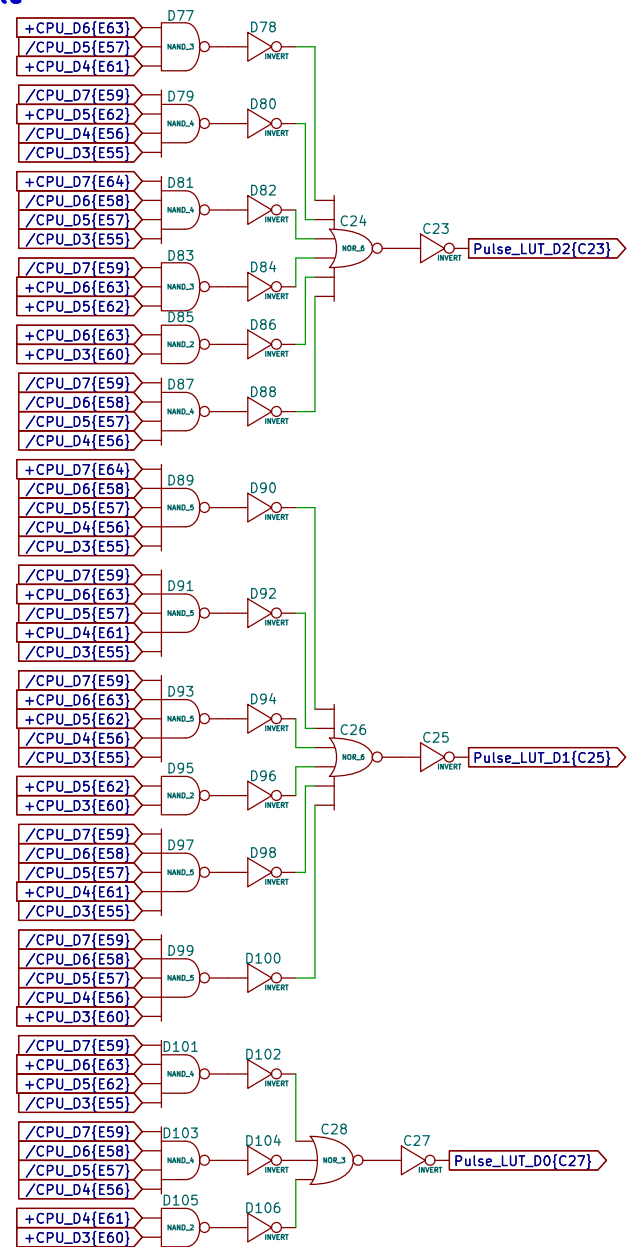
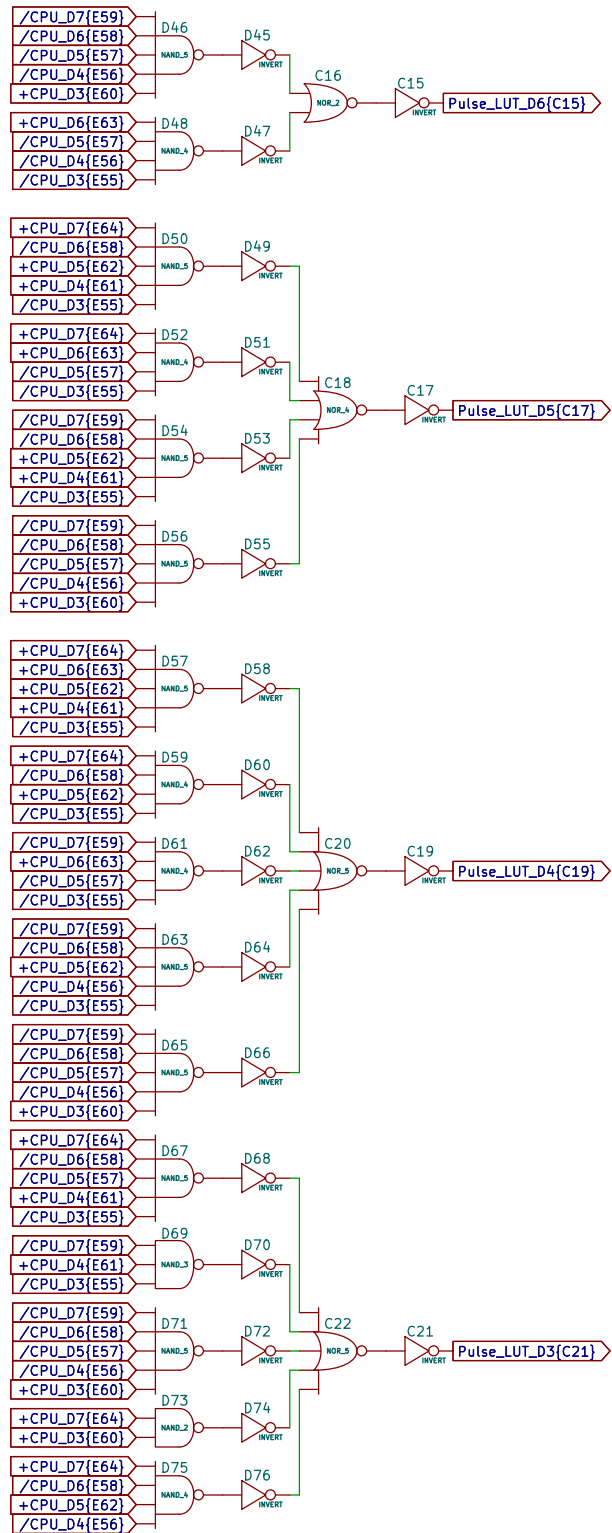


\$5104 Register ExRAM Mode

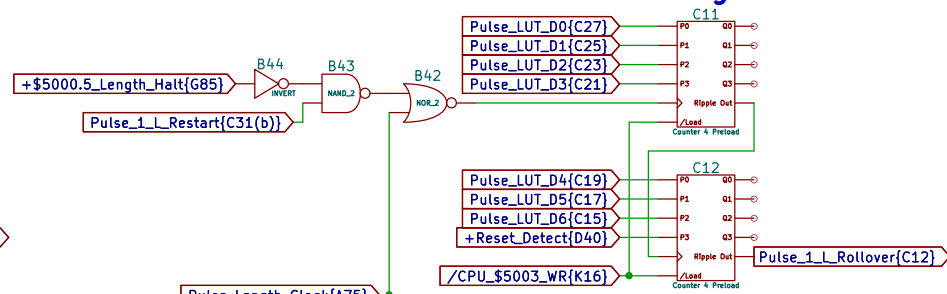




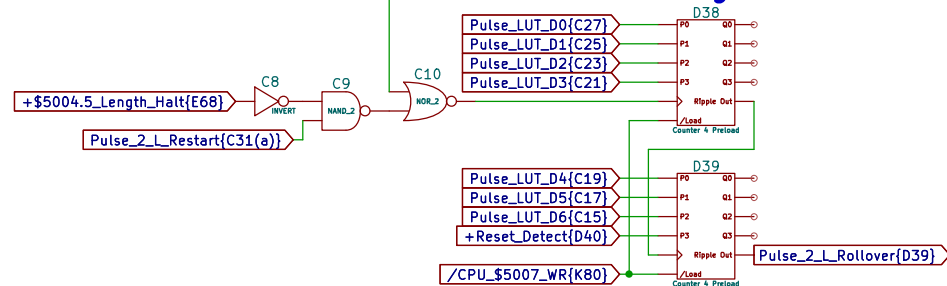
5-Bit to 7-Bit Pulse Length Lookup Table



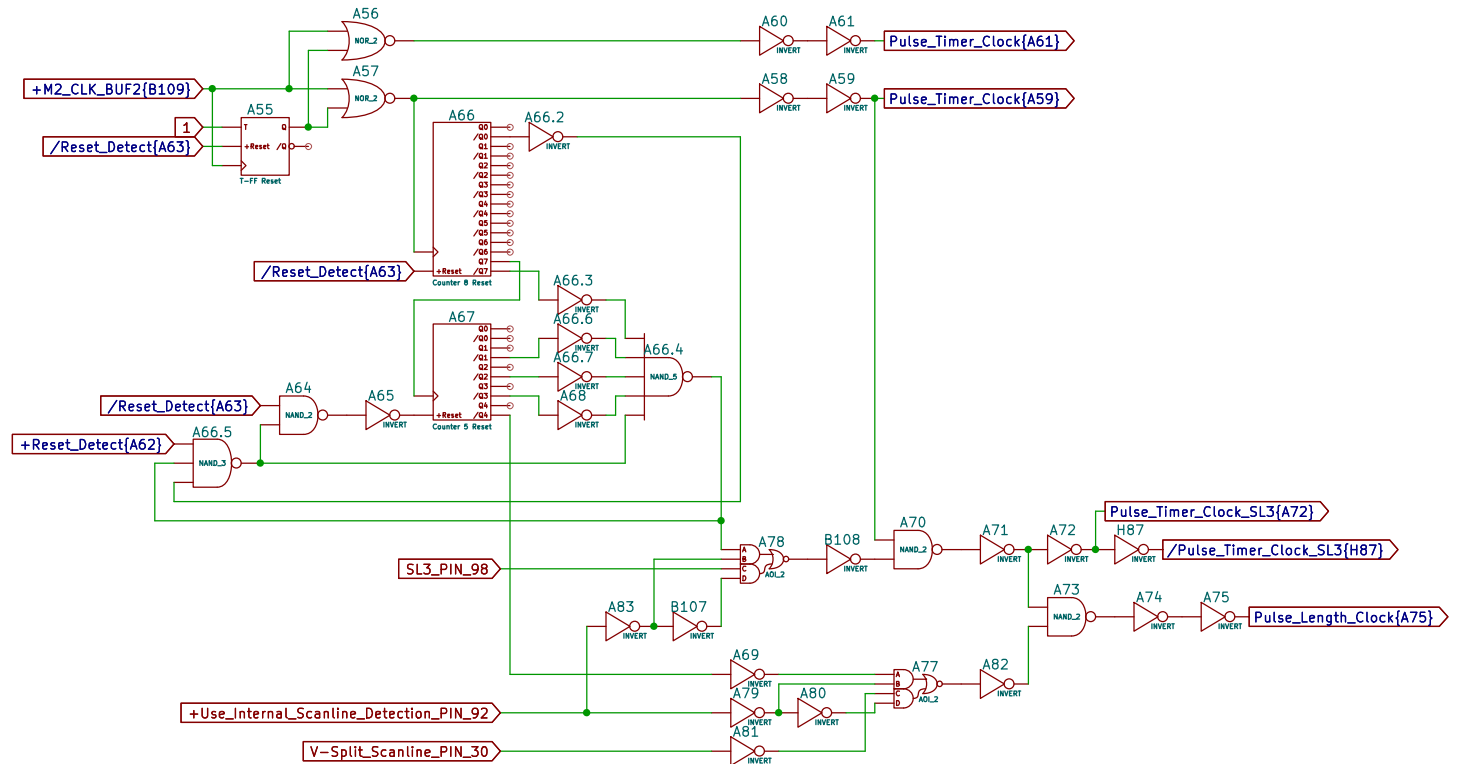
**\$5003 Register
Pulse 1 Length Counter**



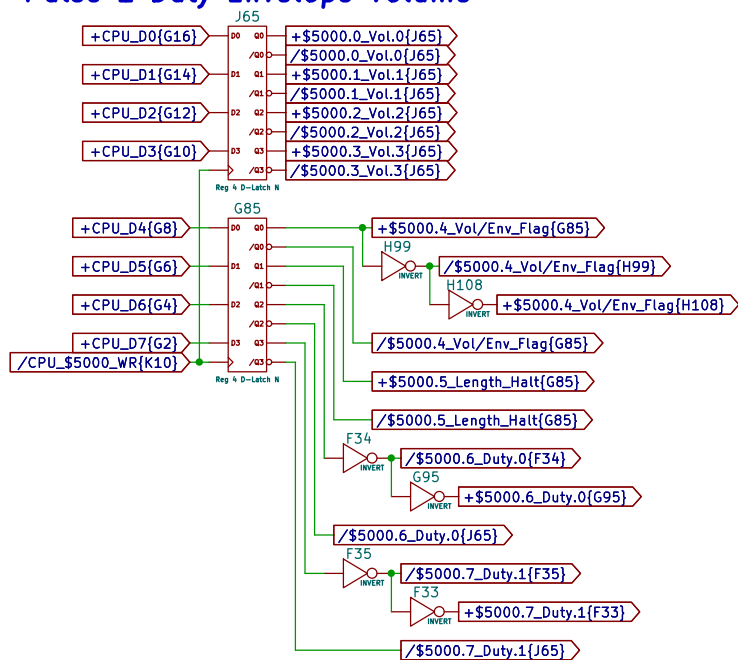
**\$5007 Register
Pulse 2 Length Counter**



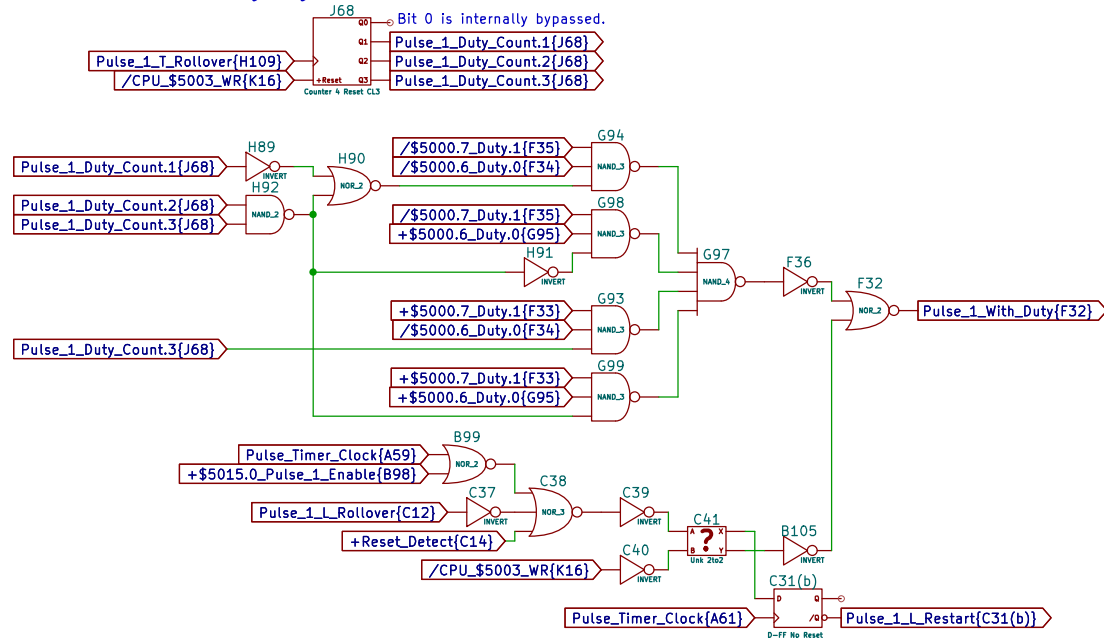
Pulse Channel Clock Source



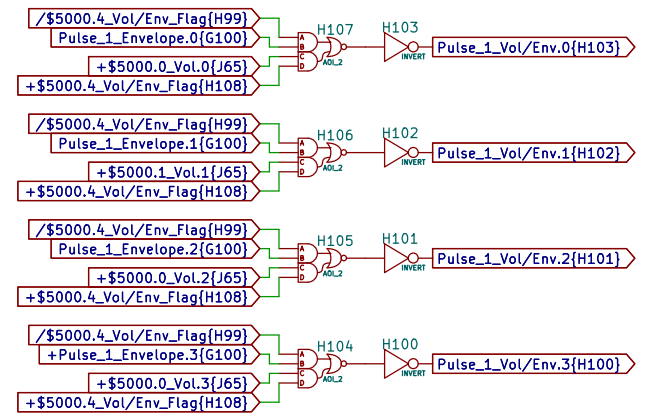
\$5000 Register Pulse 1 Duty Envelope Volume



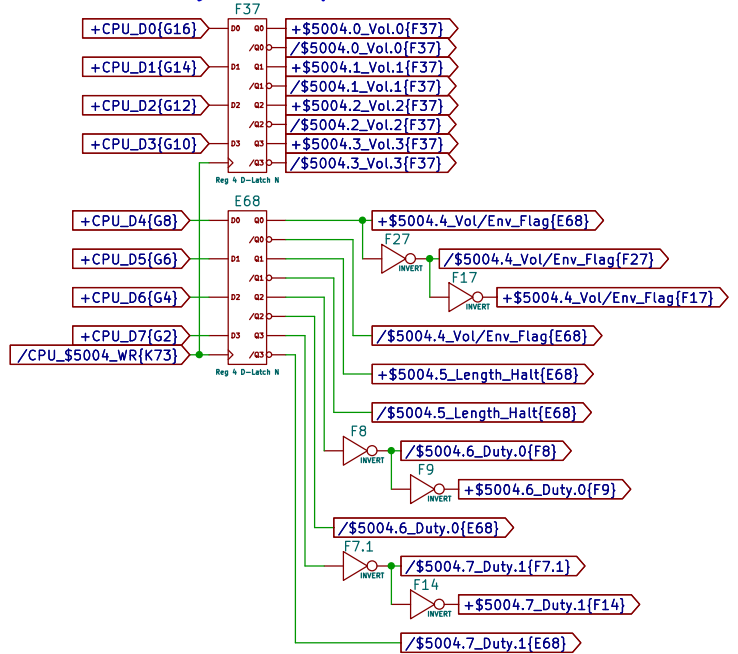
Pulse 1 Duty Cycle Counter



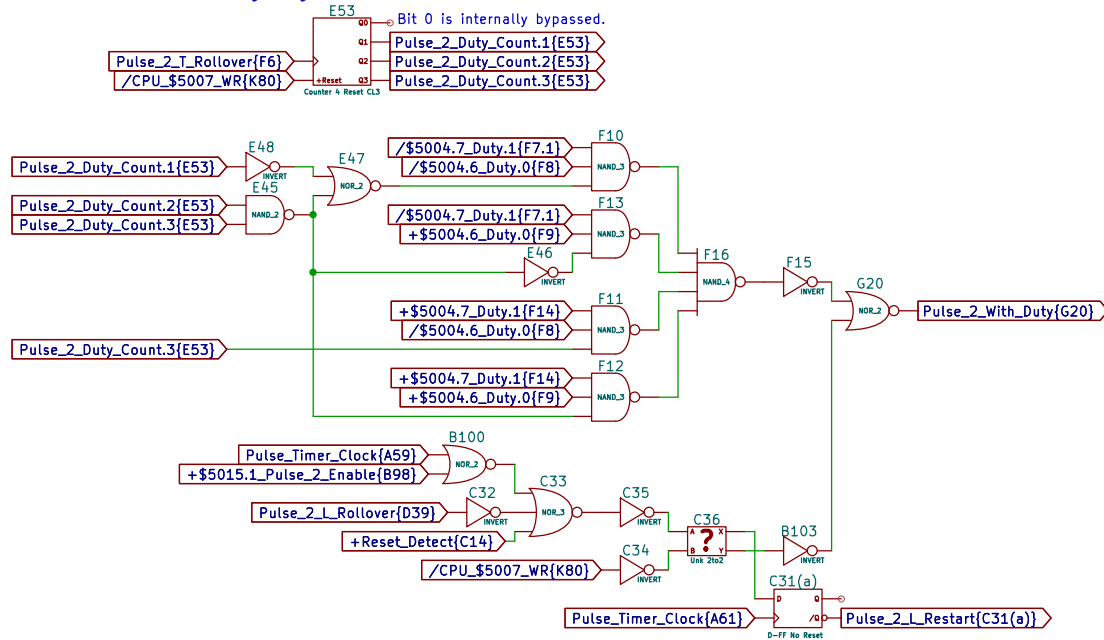
Pulse 1 Envelope/Volume Multiplexing



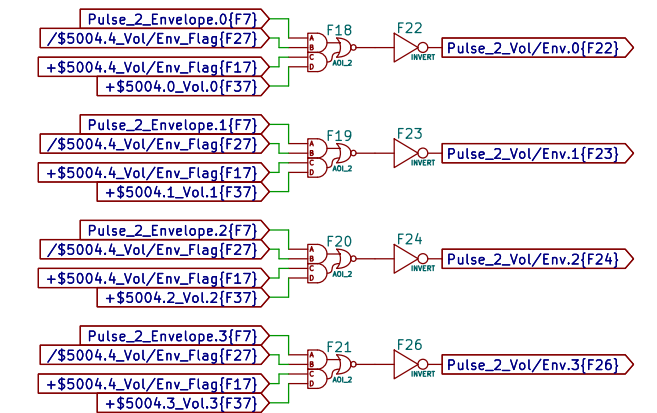
\$5004 Register Pulse 2 Duty Envelope Volume



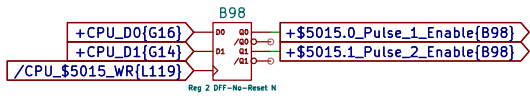
Pulse 2 Duty Cycle Counter



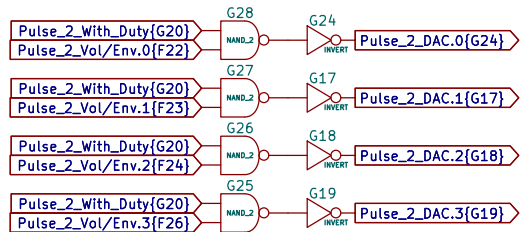
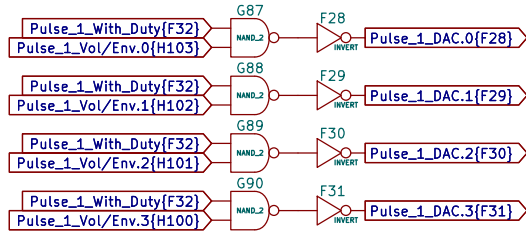
Pulse 2 Envelope/Volume Multiplexing



\$5015 Write Register Pulse 1&2 Status

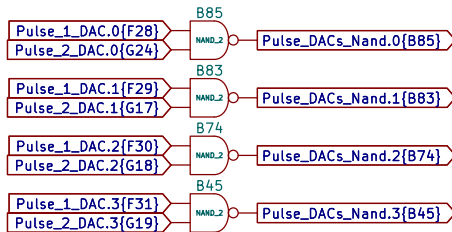
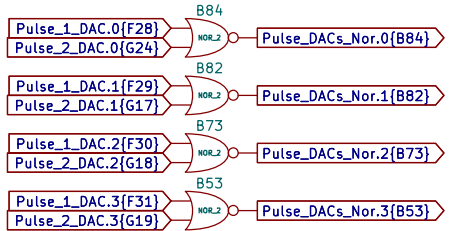


Pulse 1&2 Individual 4-bit DAC Outputs

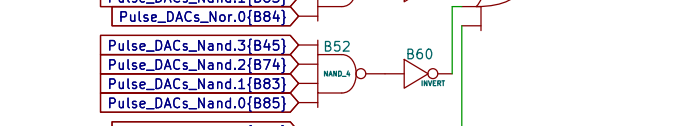
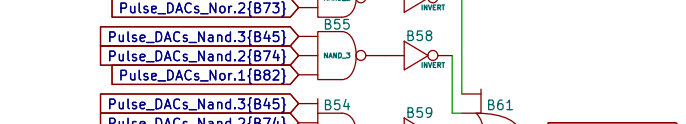
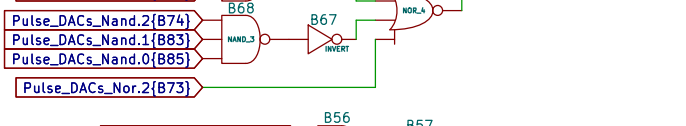
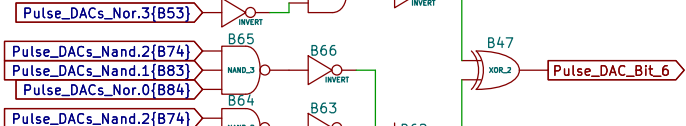
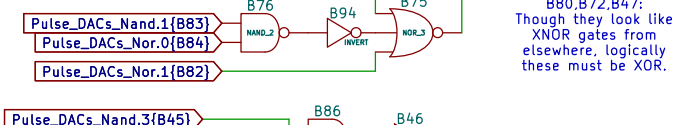
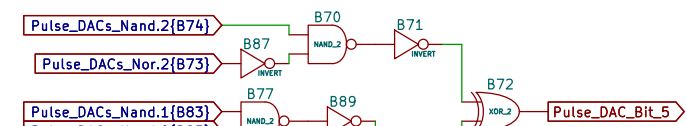
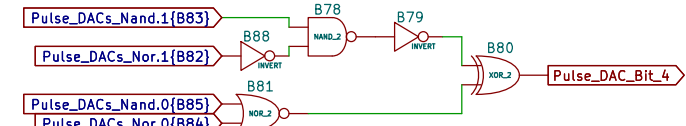
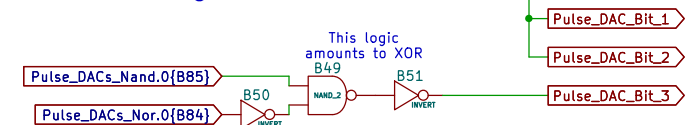


Components for 4-bit Adder

Note: The adder's logic is
verified correct in
Pulse DAC Adder Logic.xlxs



4-bit Adder Adding Pulse 1&2 DACs



Lowest 3 bits
are not used.

Pulse_DAC_Bit_0

Pulse_DAC_Bit_1

Pulse_DAC_Bit_2

Pulse_DAC_Bit_3

Pulse_DAC_Bit_4

Pulse_DAC_Bit_5

Pulse_DAC_Bit_6

Pulse_DAC_Bit_7

Pulse_DAC_Bit_8

Pulse_DAC_Bit_9

Pulse_DAC_Bit_10

Pulse_DAC_Bit_11

Pulse_DAC_Bit_12

Pulse_DAC_Bit_13

Pulse_DAC_Bit_14

Pulse_DAC_Bit_15

Pulse_DAC_Bit_16

Pulse_DAC_Bit_17

Pulse_DAC_Bit_18

Pulse_DAC_Bit_19

Pulse_DAC_Bit_20

Pulse_DAC_Bit_21

Pulse_DAC_Bit_22

Pulse_DAC_Bit_23

Pulse_DAC_Bit_24

Pulse_DAC_Bit_25

Pulse_DAC_Bit_26

Pulse_DAC_Bit_27

Pulse_DAC_Bit_28

Pulse_DAC_Bit_29

Pulse_DAC_Bit_30

Pulse_DAC_Bit_31

Pulse_DAC_Bit_32

Pulse_DAC_Bit_33

Pulse_DAC_Bit_34

Pulse_DAC_Bit_35

Pulse_DAC_Bit_36

Pulse_DAC_Bit_37

Pulse_DAC_Bit_38

Pulse_DAC_Bit_39

Pulse_DAC_Bit_40

Pulse_DAC_Bit_41

Pulse_DAC_Bit_42

Pulse_DAC_Bit_43

Pulse_DAC_Bit_44

Pulse_DAC_Bit_45

Pulse_DAC_Bit_46

Pulse_DAC_Bit_47

Pulse_DAC_Bit_48

Pulse_DAC_Bit_49

Pulse_DAC_Bit_50

Pulse_DAC_Bit_51

Pulse_DAC_Bit_52

Pulse_DAC_Bit_53

Pulse_DAC_Bit_54

Pulse_DAC_Bit_55

Pulse_DAC_Bit_56

Pulse_DAC_Bit_57

Pulse_DAC_Bit_58

Pulse_DAC_Bit_59

Pulse_DAC_Bit_60

Pulse_DAC_Bit_61

Pulse_DAC_Bit_62

Pulse_DAC_Bit_63

Pulse_DAC_Bit_64

Pulse_DAC_Bit_65

Pulse_DAC_Bit_66

Pulse_DAC_Bit_67

Pulse_DAC_Bit_68

Pulse_DAC_Bit_69

Pulse_DAC_Bit_70

Pulse_DAC_Bit_71

Pulse_DAC_Bit_72

Pulse_DAC_Bit_73

Pulse_DAC_Bit_74

Pulse_DAC_Bit_75

Pulse_DAC_Bit_76

Pulse_DAC_Bit_77

Pulse_DAC_Bit_78

Pulse_DAC_Bit_79

Pulse_DAC_Bit_80

Pulse_DAC_Bit_81

Pulse_DAC_Bit_82

Pulse_DAC_Bit_83

Pulse_DAC_Bit_84

Pulse_DAC_Bit_85

Pulse_DAC_Bit_86

Pulse_DAC_Bit_87

Pulse_DAC_Bit_88

Pulse_DAC_Bit_89

Pulse_DAC_Bit_90

Pulse_DAC_Bit_91

Pulse_DAC_Bit_92

Pulse_DAC_Bit_93

Pulse_DAC_Bit_94

Pulse_DAC_Bit_95

Pulse_DAC_Bit_96

Pulse_DAC_Bit_97

Pulse_DAC_Bit_98

Pulse_DAC_Bit_99

Pulse_DAC_Bit_100

Pulse_DAC_Bit_101

Pulse_DAC_Bit_102

Pulse_DAC_Bit_103

Pulse_DAC_Bit_104

Pulse_DAC_Bit_105

Pulse_DAC_Bit_106

Pulse_DAC_Bit_107

Pulse_DAC_Bit_108

Pulse_DAC_Bit_109

Pulse_DAC_Bit_110

Pulse_DAC_Bit_111

Pulse_DAC_Bit_112

Pulse_DAC_Bit_113

Pulse_DAC_Bit_114

Pulse_DAC_Bit_115

Pulse_DAC_Bit_116

Pulse_DAC_Bit_117

Pulse_DAC_Bit_118

Pulse_DAC_Bit_119

Pulse_DAC_Bit_120

Pulse_DAC_Bit_121

Pulse_DAC_Bit_122

Pulse_DAC_Bit_123

Pulse_DAC_Bit_124

Pulse_DAC_Bit_125

Pulse_DAC_Bit_126

Pulse_DAC_Bit_127

Pulse_DAC_Bit_128

Pulse_DAC_Bit_129

Pulse_DAC_Bit_130

Pulse_DAC_Bit_131

Pulse_DAC_Bit_132

Pulse_DAC_Bit_133

Pulse_DAC_Bit_134

Pulse_DAC_Bit_135

Pulse_DAC_Bit_136

Pulse_DAC_Bit_137

Pulse_DAC_Bit_138

Pulse_DAC_Bit_139

Pulse_DAC_Bit_140

Pulse_DAC_Bit_141

Pulse_DAC_Bit_142

Pulse_DAC_Bit_143

Pulse_DAC_Bit_144

Pulse_DAC_Bit_145

Pulse_DAC_Bit_146

Pulse_DAC_Bit_147

Pulse_DAC_Bit_148

Pulse_DAC_Bit_149

Pulse_DAC_Bit_150

Pulse_DAC_Bit_151

Pulse_DAC_Bit_152

Pulse_DAC_Bit_153

Pulse_DAC_Bit_154

Pulse_DAC_Bit_155

Pulse_DAC_Bit_156

Pulse_DAC_Bit_157

Pulse_DAC_Bit_158

Pulse_DAC_Bit_159

Pulse_DAC_Bit_160

Pulse_DAC_Bit_161

Pulse_DAC_Bit_162

Pulse_DAC_Bit_163

Pulse_DAC_Bit_164

Pulse_DAC_Bit_165

Pulse_DAC_Bit_166

Pulse_DAC_Bit_167

Pulse_DAC_Bit_168

Pulse_DAC_Bit_169

Pulse_DAC_Bit_170

Pulse_DAC_Bit_171

Pulse_DAC_Bit_172

Pulse_DAC_Bit_173

Pulse_DAC_Bit_174

Pulse_DAC_Bit_175

Pulse_DAC_Bit_176

Pulse_DAC_Bit_177

Pulse_DAC_Bit_178

Pulse_DAC_Bit_179

Pulse_DAC_Bit_180

Pulse_DAC_Bit_181

Pulse_DAC_Bit_182

Pulse_DAC_Bit_183

Pulse_DAC_Bit_184

Pulse_DAC_Bit_185

Pulse_DAC_Bit_186

Pulse_DAC_Bit_187

Pulse_DAC_Bit_188

Pulse_DAC_Bit_189

Pulse_DAC_Bit_190

Pulse_DAC_Bit_191

Pulse_DAC_Bit_192

Pulse_DAC_Bit_193

Pulse_DAC_Bit_194

Pulse_DAC_Bit_195

Pulse_DAC_Bit_196

Pulse_DAC_Bit_197

Pulse_DAC_Bit_198

Pulse_DAC_Bit_199

Pulse_DAC_Bit_200

Pulse_DAC_Bit_201

Pulse_DAC_Bit_202

Pulse_DAC_Bit_203

Pulse_DAC_Bit_204

Pulse_DAC_Bit_205

Pulse_DAC_Bit_206

Pulse_DAC_Bit_207

Pulse_DAC_Bit_208

Pulse_DAC_Bit_209

Pulse_DAC_Bit_210

Pulse_DAC_Bit_211

Pulse_DAC_Bit_212

Pulse_DAC_Bit_213

Pulse_DAC_Bit_214

Pulse_DAC_Bit_215

Pulse_DAC_Bit_216

Pulse_DAC_Bit_217

Pulse_DAC_Bit_218

Pulse_DAC_Bit_219

Pulse_DAC_Bit_220

Pulse_DAC_Bit_221

Pulse_DAC_Bit_222

Pulse_DAC_Bit_223

Pulse_DAC_Bit_224

Pulse_DAC_Bit_225

Pulse_DAC_Bit_226

Pulse_DAC_Bit_227

Pulse_DAC_Bit_228

Pulse_DAC_Bit_229

Pulse_DAC_Bit_230

Pulse_DAC_Bit_231

Pulse_DAC_Bit_232

Pulse_DAC_Bit_233

Pulse_DAC_Bit_234

Pulse_DAC_Bit_235

Pulse_DAC_Bit_236

Pulse_DAC_Bit_237

Pulse_DAC_Bit_238

Pulse_DAC_Bit_239

Pulse_DAC_Bit_240

Pulse_DAC_Bit_241

Pulse_DAC_Bit_242

Pulse_DAC_Bit_243

Pulse_DAC_Bit_244

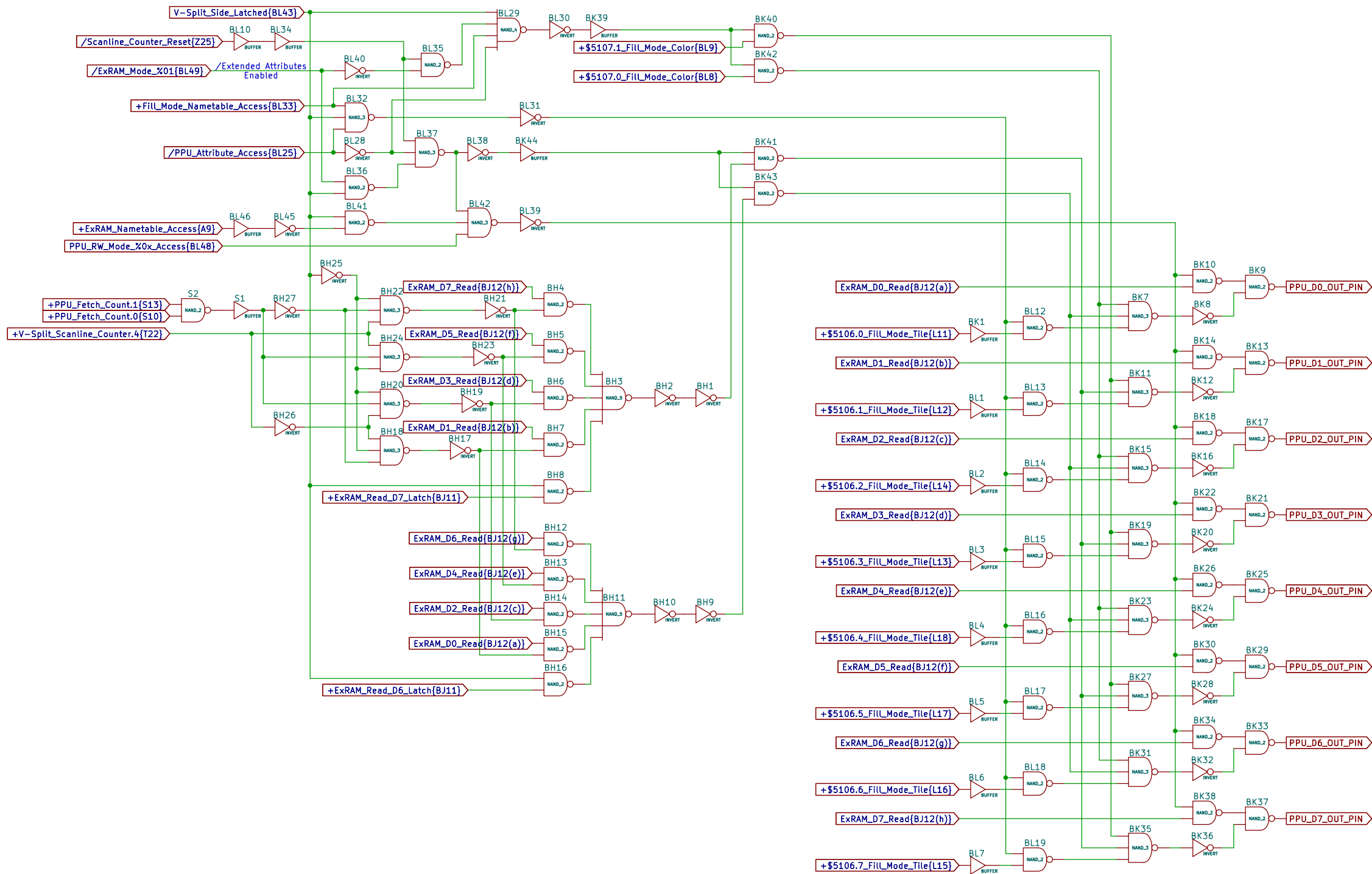
Pulse_DAC_Bit_245

Pulse_DAC_Bit_246

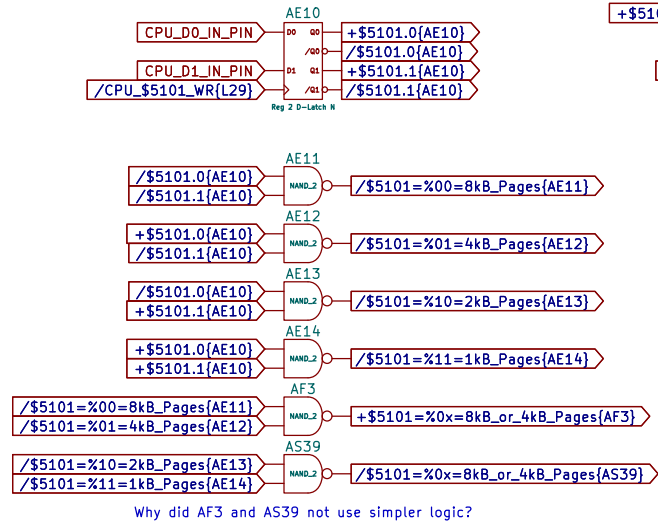
Pulse_DAC_Bit_247

Pulse_DAC_Bit_248

Pulse_DAC_Bit_249

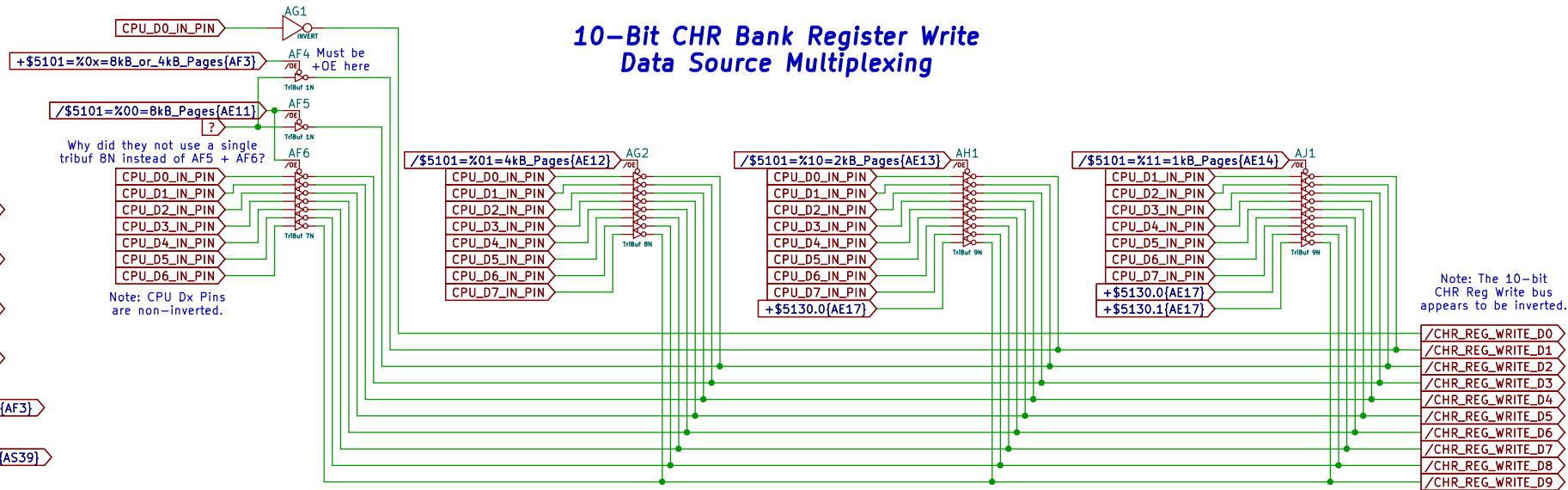


\$5101 Register CHR Mode



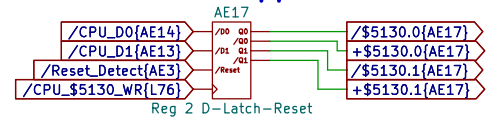
Why did AF3 and AS39 not use simpler logic?

10-Bit CHR Bank Register Write Data Source Multiplexing

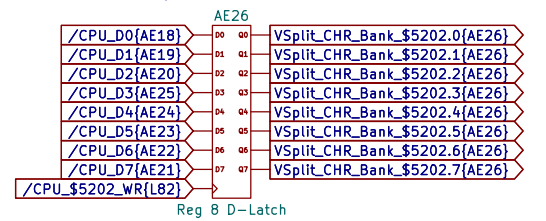


Note: The 10-bit
CHR Reg Write bus
appears to be inverted.

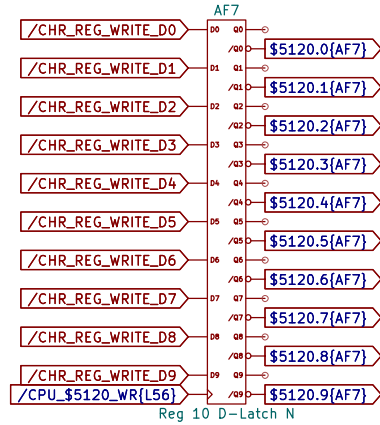
\$5130 Register
CHR Bank Upper Bits



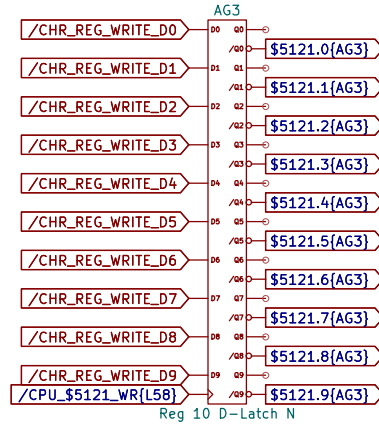
**\$5202 Register
V-Split CHR Bank**



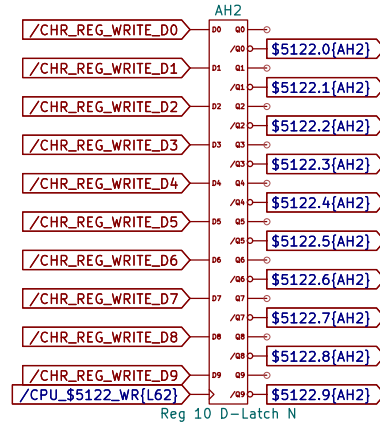
\$5120 Register



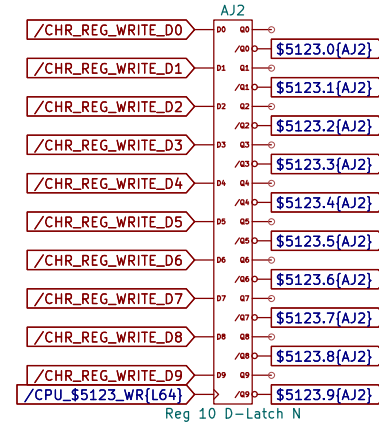
\$5121 Register



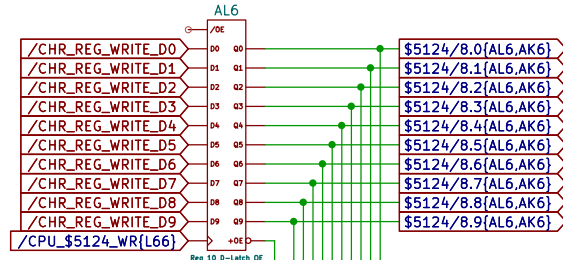
\$5122 Register



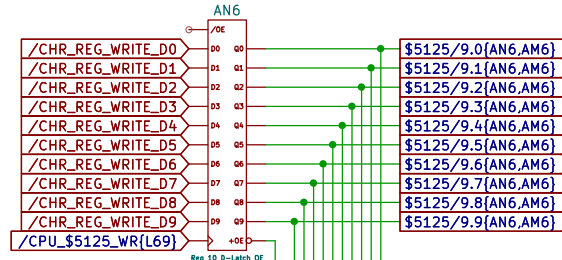
\$5123 Register



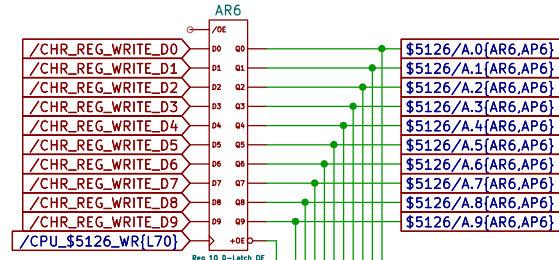
\$5124 Register



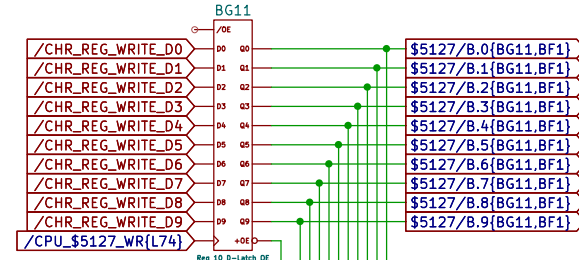
\$5125 Register



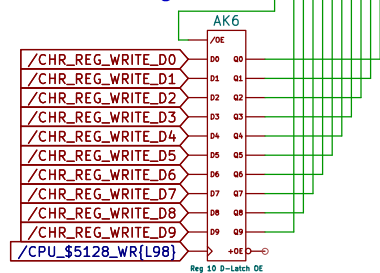
\$5126 Register



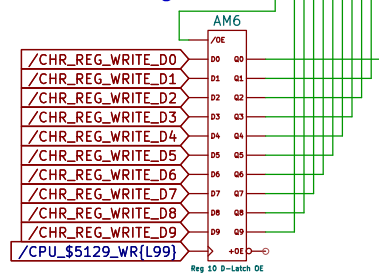
\$5127 Register



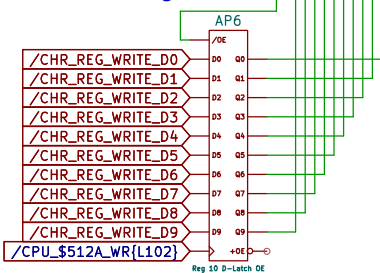
\$5128 Register



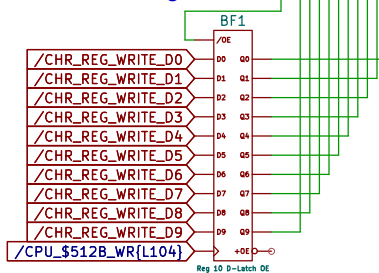
\$5129 Register

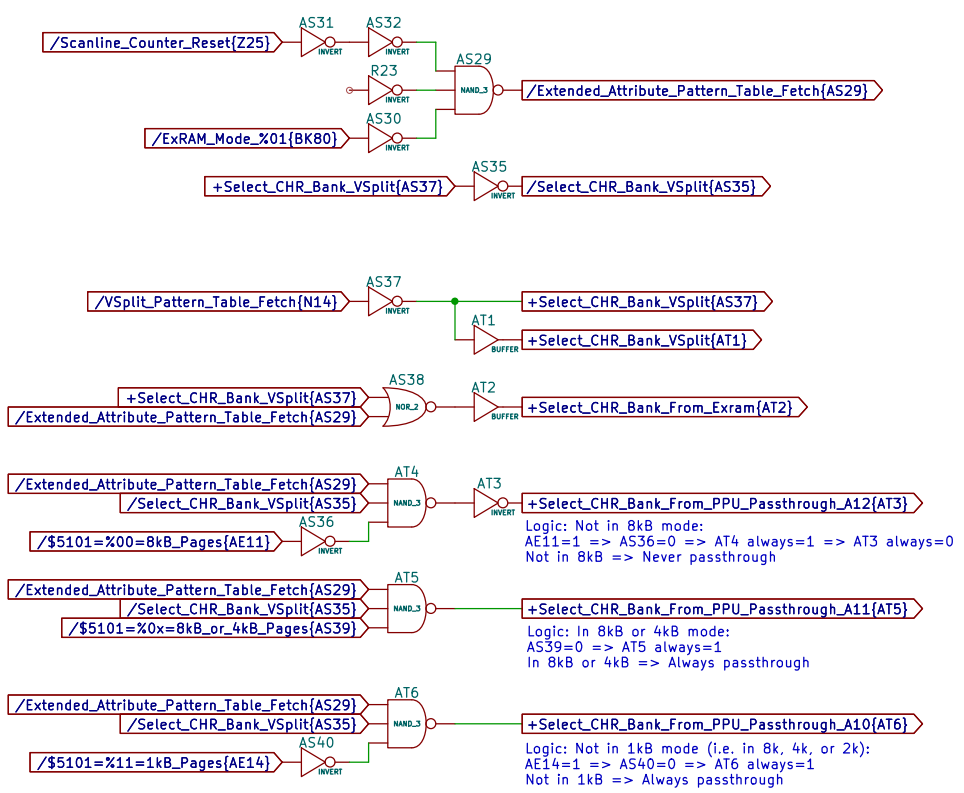
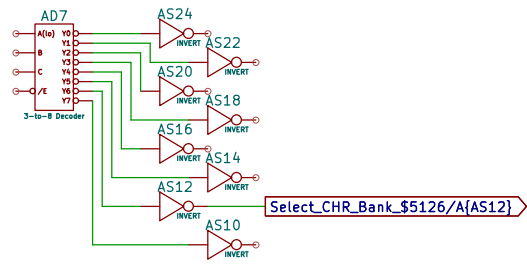
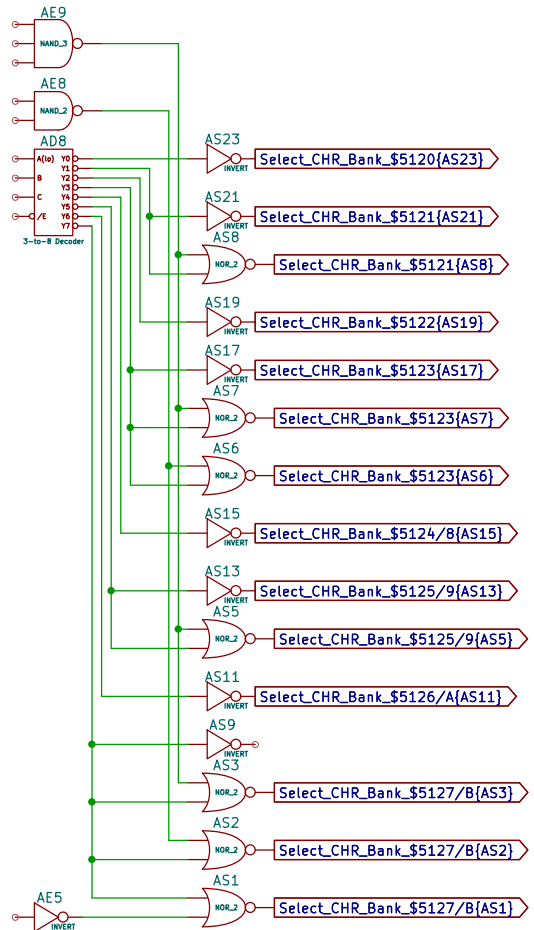


\$512A Register



\$512B Register





Logic: Not in 8kB mode:
AE11=1 => AS36=0 => AT4 always=1 => AT3 always=0
Not in 8kB => Never passthrough

Logic: In 8kB or 4kB mode:
AS39=0 => AT5 always=1
In 8kB or 4kB => Always passthrough

Logic: Not in 1kB mode (i.e. in 8k, 4k, or 2k):
AE14=1 => AS40=0 => AT6 always=1
Not in 1kB => Always passthrough

